

세션 1 · Redis 운영

Cache 패턴 · Persistence · Eviction 정책 · Replication · Sentinel · 자동 전환

대상 버전

Redis 8.10.x

OS

Ubuntu 24.04 LTS

실습 환경

Azure

01

인프라 구축

스택 구성

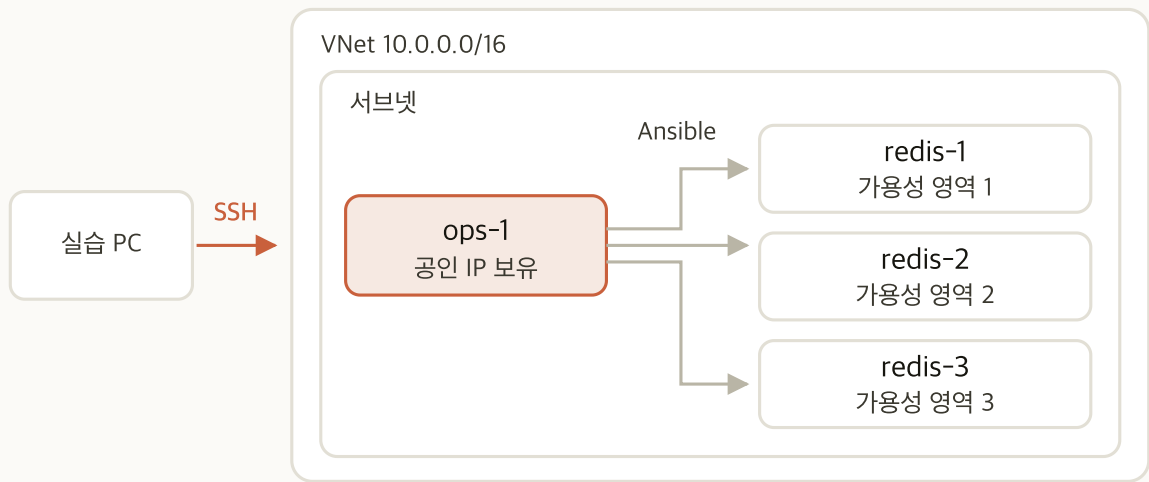
Terraform 실행

자원 확인

네트워크 규칙

노드 접속

01-redis 스택 구성



항목	내용
redis-1	Standard_E2bds_v5 (2 vCPU / 16 GiB)
redis-2	Redis와 Sentinel
redis-3	
ops-1	Standard_D2ds_v5 (2 vCPU / 8 GiB) Ansible 실행, 외부 접속 지점
데이터 디스크	64 GiB · 3,000 IOPS · 125 MB/s Premium SSD v2
OS	전 노드 Ubuntu 24.04 LTS

관리형 서비스를 쓰지 않습니다.
 VM에 직접 설치해야 설정값과 장애 동작을 볼 수 있고, 이후 온프레미스에서도 같은 절차가 그대로 적용됩니다.

인스턴스 스펙 근거

항목	값	근거
인스턴스	Standard_E2bds_v5 (2 vCPU / 16 GiB)	메모리 최적화 계열
vCPU	2	명령 실행이 단일 스레드이므로 코어를 늘려도 비례하지 않음
메모리	16 GiB	데이터 용량이 곧 인스턴스 크기
계열 제외	Burstable(B 시리즈)	크레딧 소진 시 성능이 강등됨

Redis에서 확장의 축은 CPU가 아니라 메모리입니다.
단일 노드 메모리가 곧 이 인스턴스의 용량 한계입니다.

스택 파일 구성

파일	내용
<code>versions.tf</code>	Provider와 버전 제약
<code>variables.tf</code>	구독 ID, 리소스 그룹, 리전, 접속 허용 IP, SSH 공개 키 경로
<code>main.tf</code>	VNet, NSG, VM 4대, 데이터 디스크, 실습용 키 쌍
<code>outputs.tf</code>	ops 노드 공인 IP, 서비스 노드 사설 IP, 접속 명령

이 스택의 코드가 곧 인프라 사양입니다.

변수 파일 작성

```
subscription_id      = "000000000-0000-0000-0000-000000000000"
resource_group_name  = "rkm-<계정 이름>-redis-rg"
location             = "koreacentral"
allowed_cidrs        = ["xxx.xxx.xxx.xxx/32"]
ssh_public_key_path  = "~/.ssh/rkm.pub"
```

- **subscription_id**: `az account show --query id -o tsv`
- **resource_group_name**: `rkm-<전달받은 계정 이름>-redis-rg`. 형식이 다르면 `apply` 전에 멈춤
- **allowed_cidrs**: `curl -s ifconfig.me` 로 확인한 자기 PC 공인 IP

구독 ID와 키는 저장소에 커밋하지 않습니다.

`session.tfvars` 는 `.gitignore` 에 들어 있습니다. 예제 파일만 저장소에 둡니다.

Terraform 실행

- 1 `cd lab/terraform/01-redis` 로 이동
- 2 `cp session.tfvars.example session.tfvars` 후 변수 파일의 다섯 값 입력
- 3 `terraform init` 실행
- 4 `terraform plan -var-file=session.tfvars` 로 생성될 자원 목록 확인
- 5 `terraform apply -var-file=session.tfvars` 실행
- 6 `terraform output` 으로 ops 노드 공인 IP 확인

완료 판정

`Apply complete!` 로 끝나고 `terraform output` 에 ops 노드 공인 IP가 나오면 완료입니다.

plan 출력 확인 항목

```
Plan: 27 to add, 0 to change, 0 to destroy.
```

```
Changes to Outputs:
```

```
+ ops_public_ip      = (known after apply)
+ redis_private_ips  = (known after apply)
+ ssh_command        = (known after apply)
```

- **추가 자원 수:** 리소스 그룹과 VM 4대에 딸린 NIC·디스크·네트워크 자원, 실습용 키 쌍 포함
- **삭제 0:** 기존 자원을 지우지 않음

plan 에서 삭제 항목이 보이면 실행하지 마세요.

다른 스택의 상태 파일을 가리키고 있을 수 있습니다.

포털 확인 항목

확인 대상	포털 메뉴	확인 내용
자원 목록	리소스 그룹	VM 4대, 디스크, NIC, NSG, VNet
VM과 영역	가상 머신	redis-1·redis-2·redis-3 이 영역 1·2·3에 하나씩
네트워크	가상 네트워크	10.0.0.0/16 안의 서브넷 하나
인바운드 규칙	네트워크 보안 그룹	22·3000·9090이 현재 PC 공인 IP로 제한
디스크	디스크	64 GiB, 3,000 IOPS, 125 MB/s

생성 결과 확인

☐	Name ↑		Type	Location	
☐	 ops-1	...	Virtual machine	Korea Central	
☐	 ops-1-data	...	Disk	Korea Central	
☐	 ops-1-nic	...	Network Interfa...	Korea Central	
☐	 ops-1-pip	...	Public IP address	Korea Central	
☐	 ops-1_OsDisk_1_569f4444ad0f46		Disk	Korea Central	
☐	 redis-1	...	Virtual machine	Korea Central	

리소스 그룹: VM 4대와 함께 만들어진 디스크·NIC

☐	Name ↑		Resource Group	Location	Status	Operating system	Size	Disks	Availability zone
☐	 ops-1	...	rkm-verify-rg	Korea Central	Running	Linux	Standard_D2ds_v5	2	1
☐	 redis-1	...	rkm-verify-rg	Korea Central	Running	Linux	Standard_E2bds_v5	2	1
☐	 redis-2	...	rkm-verify-rg	Korea Central	Running	Linux	Standard_E2bds_v5	2	2
☐	 redis-3	...	rkm-verify-rg	Korea Central	Running	Linux	Standard_E2bds_v5	2	3

가상 머신: 가용성 영역이 1·2·3으로 흩어져 있음

네트워크와 디스크 확인

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓	Destination ↑↓	Action ↑↓
Inbound Security Rules						
100	allow-intra-subnet	Any	Any	10.0.1.0/24	10.0.1.0/24	✓ Allow
200	allow-ssh	22	Tcp		Any	✓ Allow
300	allow-3000	3000	Tcp		Any	✓ Allow
301	allow-9090	9090	Tcp		Any	✓ Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	✓ Allow
65001	AllowAzureLoadBalancerInBound	Any	Any	AzureLoadBalancer	Any	✓ Allow
65500	DenyAllInBound	Any	Any	Any	Any	✗ Deny

네트워크 보안 그룹: 22·3000·9090 규칙의 소스가 현재 PC 공인 IP 하나

Storage type ⓘ
Premium SSD v2 (locally-redundant storage)
Why are some options disabled? ⓘ

Max size	Disk tier	Provisioned IOPS	Provisioned throughput	Max Shares
65536 GiB	P	80000	2000	15

Custom disk size (GiB) * ⓘ

Disk IOPS *
 ✓

Disk throughput (MB/s) *
 ✓

데이터 디스크: 64 GiB · 3,000 IOPS · 125 MB/s

공인 IP가 바뀌면 접속이 끊깁니다. session.tfvars 의 allowed_cidrs 를 고친 뒤 apply 를 다시 실행합니다.

ops 노드 접속

Git Bash

```
ssh -i ~/.ssh/rkm azureuser@<ops 공인 IP>
```

- **대안:** PuTTY. 이 경우 키를 `.ppk` 로 변환
- **접속 후 위치:** `ops-1`. 서비스 노드는 사실 IP로만 접근

서비스 노드에는 공인 IP가 없습니다.
접속은 ops 노드를 거칩니다. 방법은 다음 장에 있습니다.

서비스 노드 접근 방식

```
# ops 노드에서 서비스 노드로 직접 접속
ssh 10.0.1.6
```

```
# 세 노드에 같은 명령을 보낼 때는 Ansible 을 쓴다
```

이 세션의 명령은 모두 ops 노드에서 실행합니다.

세 노드를 오가지 않아도 되고, 같은 명령을 세 노드에 한 번에 보낼 수 있습니다.

02

Redis 구성

인벤토리

redis.conf

Sentinel

exporter

기동 확인

플레이북 구조

Role	하는 일
common	디스크 마운트, <code>vm.overcommit_memory=1</code> , THP 비활성, 시간 동기화
redis	Redis 8.10.x 설치, <code>redis.conf</code> 배포
sentinel	Sentinel 설치와 <code>sentinel.conf</code> 배포. 기동은 하지 않음
monitoring	exporter 설치, ops 노드에 Prometheus·Grafana 기동

Ansible은 클라우드에 의존하지 않습니다.

같은 플레이북이 온프레미스 서버에도 그대로 적용됩니다. 클라우드 의존은 앞의 인프라 생성까지입니다.

인벤토리: 노드 목록

```
[redis]
redis-1 ansible_host=10.0.1.5 redis_role=master
redis-2 ansible_host=10.0.1.6 redis_role=replica
redis-3 ansible_host=10.0.1.7 redis_role=replica
```

```
[ops]
ops-1 ansible_connection=local
```

```
[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

- 채우는 주체: Terraform. 사설 IP를 채운 파일을 ops 노드에 배치
- 받는 시점: ops 노드가 부팅할 때 실습 저장소에서 플레이북과 적재 스크립트(`~/loadgen/fill.sh`)를 함께 내려받음
- `[ops]` : `local` 연결. 플레이북을 돌리는 노드 자신
- 접속 방식: ops 노드의 개인 키로 서비스 노드에 SSH

인벤토리: 공용 변수

```
# Sentinel 이 감시할 master. 위 redis_role=master 노드와 같아야 함
redis_master_host=10.0.1.5
redis_port=6379
sentinel_port=26379

# master 가 정지했다고 합의할 최소 Sentinel 수. Sentinel 3대이므로 2
sentinel_quorum=2
```

- **redis_master_host** : Terraform 이 채우고 Ansible 이 **sentinel.conf** 에 넣음
- **sentinel_quorum** : 2. master 정지에 합의할 최소 Sentinel 수
- 쓰이는 곳: **redis.conf** 와 **sentinel.conf** 템플릿

redis.conf 주요 설정

```
protected-mode no
port {{ redis_port }}

# systemd 에 준비 완료를 알린다.
# Ubuntu 의 redis-server.service 는 Type=notify 라 이 신호를 기다린다.
# 빠뜨리면 기동은 되지만 systemd 가 90초를 기다리다 시간 초과로 죽인다.
supervised systemd
```

- 배포 위치: 세 Redis 노드의 `/etc/redis/redis.conf`. Ansible 플레이북이 배포하므로 직접 쓰지 않음
- `protected-mode no`: ops 노드에서 `redis-cli -h` 로 원격 접속하는 구조라 꺼 둠. 공인 IP 없음과 NSG 로 두 겹 차단
- `port`: 인벤토리의 `redis_port` 가 들어감
- `supervised systemd`: 준비 완료를 systemd 에 알림. 빠뜨리면 90초 뒤 시간 초과로 죽음

플레이북 실행

- 1 `cd ~/ansible`
- 2 `ansible redis -m ping` 으로 서비스 노드 3대 연결 확인
- 3 `ansible-playbook play-redis.yml` 실행
- 4 `ansible redis -a "redis-cli PING"` 으로 세 노드 응답 확인
- 5 `ansible redis -a "redis-cli CONFIG GET port"` 로 인벤토리의 `redis_port` 가 적용됐는지 확인

```
PLAY RECAP *****
redis-1 : ok=23 changed=15 unreachable=0 failed=0 skipped=8
redis-2 : ok=23 changed=15 unreachable=0 failed=0 skipped=8
redis-3 : ok=23 changed=15 unreachable=0 failed=0 skipped=8
ops-1   : ok=11 changed=10 unreachable=0 failed=0 skipped=2
```

4대 모두 failed=0 이고 각 노드에서 redis-cli PING 이 PONG 이면 완료입니다. 2단계는 ops에서 서비스 노드로 가는 SSH 연결을 확인하는 것이므로 redis 그룹만 지정합니다. skipped 는 monitoring Role에서 적용 대상이 아닌 task입니다.

명령 전달 방식

방식	언제 쓰는가
<code>redis-cli -h 10.0.1.5 <명령></code>	기본. 명령 하나를 치고 결과만 볼 때, 셸 명령과 번갈아 칠 때
<code>redis-cli -h 10.0.1.5</code>	명령을 여러 개 이어서 칠 때. 프롬프트가 <code>10.0.1.5:6379></code> 로 바뀌고 <code>exit</code> 로 나옴
<code>ansible redis -a "redis-cli <명령>"</code>	세 노드에 같은 명령을 걸 때. 노드마다 결과가 따로 나옴
<code>ansible <노드> -a "<셸 명령>" --become</code>	노드의 파일과 프로세스를 다룰 때. <code>ls · systemctl</code> 처럼 Redis 밖의 것

넷 다 ops 노드에서 실행합니다.

서비스 노드에 직접 들어가지 않습니다. `-h` 를 빼면 `redis-cli` 가 자기 자신에게 붙는데 ops 노드에는 Redis 가 없어 연결이 거부됩니다.

Prometheus·Grafana 지표 확인

- 1 ops 노드의 9090 포트로 Prometheus 접속. **Status** → **Targets** 에서 exporter 가 전부 **UP** 인지 확인. 추가 직후에는 **UNKNOWN** 으로 표시됩니다
- 2 같은 노드의 3000 포트로 Grafana 접속. 초기 계정은 **admin** / **admin**
- 3 **실습** 폴더의 대시보드에서 세 노드의 CPU·메모리와 Redis 메모리 사용량·연결 수 확인

node (3/3 up) show less	
Endpoint	State
http://10.0.1.5:9100/metrics	UP
http://10.0.1.6:9100/metrics	UP
http://10.0.1.7:9100/metrics	UP
redis (3/3 up) show less	
Endpoint	State
http://10.0.1.5:9121/metrics	UP
http://10.0.1.6:9121/metrics	UP
http://10.0.1.7:9121/metrics	UP

Prometheus: **node** 와 **redis** 가 각각 3/3 UP



Grafana: Redis 지표 두 패널. 위에 노드 CPU·메모리 패널이 함께 있음

여기서는 지표가 보이는 것까지만 확인합니다. `rate()` 를 쓰는 패널은 표본이 둘 쌓여야 선이 그려집니다. 3000과 9090은 현재 PC의 공인 IP로만 열려 있습니다.

03

Redis 주요 개념

단일 스레드

자료구조

명령 복잡도

네트워크 왕복

기본 명령 실습

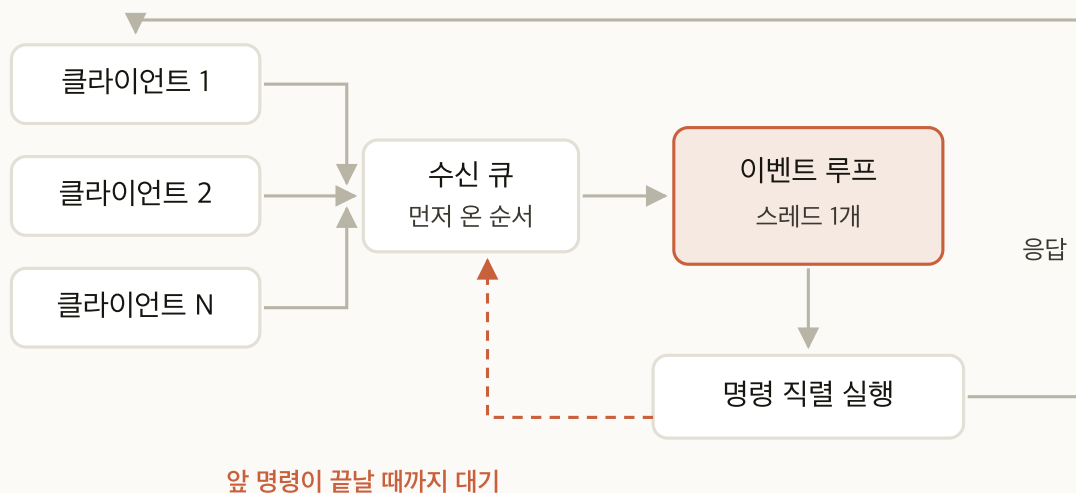
In-memory 데이터 구조 서버

- **저장 위치:** 메모리. 디스크는 Persistence 용도
- **저장 단위:** 단순 문자열이 아니라 서버가 이해하는 자료구조
- **연산 위치:** 값을 가져와 계산하지 않고 서버에서 연산

자료구조 선택이 곧 성능입니다.

같은 데이터를 String으로 넣는지 Hash로 넣는지에 따라 명령의 시간 복잡도가 달라집니다.

단일 스레드 이벤트 루프



- **명령 실행:** 하나의 스레드에서 직렬 처리
- **I/O 스레드:** 읽기·쓰기 처리에만 쓰임. 명령 실행은 여전히 스레드 하나
- **결과:** 명령 사이에 경합이 없어 원자성을 보장하기 쉬움

느린 명령 하나가 서버 전체를 멈춥니다.

대기 중인 모든 클라이언트가 그 명령이 끝날 때까지 기다립니다. 이 세션에서 다루는 위험의 상당수가 여기서 비롯됩니다.

기본 자료구조

자료구조	쓰임
String	캐시 값, 카운터
Hash	객체 한 건의 필드 묶음
List	큐, 최근 항목
Set	중복 없는 집합, 교집합 연산
Sorted Set	순위, 시간 범위 조회
Stream	추가 전용 로그, Consumer Group

String: 캐시 값과 카운터

```
SET user:1001:profile '{"name":"김민준","tier":"gold"}'
SETEX session:ab12 1800 "user:1001"
INCR page:home:views
GET user:1001:profile
```

- **담는 것:** 직렬화한 객체 한 건, 세션 토큰, 카운터
- **고르는 기준:** 값 전체를 통째로 읽고 쓸 때
- **대표 명령:** SET GET SETEX INCR MGET

값의 일부만 자주 고친다면 String이 아니라 Hash입니다.

String은 필드 단위 갱신이 없어서 한 글자를 바꿔도 값 전체를 다시 써야 합니다.

Hash: 객체 한 건의 필드 묶음

```
HSET user:1001 name "김민준" tier "gold" points 320
HINCRBY user:1001 points 50
HGET user:1001 tier
HMGET user:1001 name points
```

- **담는 것:** 필드가 여럿인 객체 한 건
- **고르는 기준:** 객체의 일부 필드만 읽거나 고칠 때
- **대표 명령:** HSET HGET HMGET HINCRBY HSCAN

HGETALL 은 필드가 많을수록 위험합니다.

모든 필드를 한 번에 반환하므로 응답을 만드는 동안 서버가 다른 명령을 처리하지 못합니다. 필요한 필드만 **HMGET** 으로 가져옵니다.

List: 큐와 최근 항목

```
LPUSH queue:mail "job:5821"
RPOP queue:mail
LPUSH user:1001:recent "item:77"
LTRIM user:1001:recent 0 9
LRANGE user:1001:recent 0 -1
```

- **담는 것:** 처리 대기 작업, 최근 본 항목
- **고르는 기준:** 넣은 순서 자체가 의미를 가질 때
- **대표 명령:** LPUSH RPOP LRANGE LTRIM BRPOP

길이를 제한하지 않으면 List는 계속 늘어납니다.

최근 N건만 남기는 용도라면 넣을 때마다 LTRIM 으로 잘라 냅니다.

Set: 중복 없는 집합

```
SADD article:99:tags redis cache ops
SADD user:1001:liked item:77 item:81
SISMEMBER user:1001:liked item:77
SINTER article:99:tags article:120:tags
SCARD user:1001:liked
```

- **담는 것:** 태그, 좋아요 목록, 중복을 허용하지 않는 식별자 묶음
- **고르는 기준:** 포함 여부 확인과 집합 연산이 필요할 때
- **대표 명령:** SADD SISMEMBER SINTER SCARD SSCAN

SMEMBERS 는 전체를 한 번에 반환합니다.

원소가 많으면 **SSCAN** 으로 나눠 읽습니다. 개수만 필요하면 **SCARD** 를 씁니다.

Sorted Set: 순위와 시간 범위

```
ZADD rank:weekly 320 user:1001
ZINCRBY rank:weekly 50 user:1001
ZRANGE rank:weekly 0 9 REV WITHSCORES
ZADD events 1756180000 "login:1001"
ZRANGE events 1756100000 1756200000 BYSCORE
```

- **담는 것:** 점수를 함께 저장한 항목. 순위표, 시각을 점수로 둔 이벤트
- **고르는 기준:** 정렬된 상태를 유지하며 범위로 꺼낼 때
- **대표 명령:** ZADD ZINCRBY ZRANGE ... REV ZRANGE ... BYSCORE ZREM

점수 자리에 UNIX 시각을 넣으면 시간 범위 조회가 됩니다.

순위표와 시계열이라는 서로 다른 두 용도가 같은 자료구조에서 나옵니다.

Stream: 추가 전용 로그

```
XADD orders * order_id 5821 status paid
XGROUP CREATE orders billing 0
XREADGROUP GROUP billing worker1 COUNT 10 STREAMS orders >
XACK orders billing 1756180000-0
```

- **담는 것:** 순서가 있는 이벤트 기록
- **고르는 기준:** 여러 컨슈머가 나눠 처리하고 처리 완료를 표시해야 할 때
- **대표 명령:** XADD XREADGROUP XACK XLEN XTRIM

List와 달리 읽어도 사라지지 않습니다.

Consumer Group 의 진행 위치를 서버가 기억하므로 재시작해도 이어서 처리합니다.

주의가 필요한 명령

명령	하는 일	복잡도와 위험
KEYS pattern	패턴에 맞는 키를 전부 찾아 한 번에 반환	O(N). 키 전체를 훑는 동안 다른 명령이 대기
SMEMBERS	Set의 모든 원소를 한 번에 반환	O(N). 원소 수만큼 응답을 만드는 동안 점유
HGETALL (큰 Hash)	Hash의 모든 필드와 값을 반환	O(N). 필드가 많을수록 점유가 길어짐
FLUSHALL	모든 DB의 데이터를 즉시 삭제	O(N). 삭제 자체가 오래 걸리고 되돌릴 수 없음

FLUSHALL 은 확인 절차 없이 즉시 실행되고 되돌릴 수 없습니다.
 실습에서 데이터를 비울 때는 대상 DB를 지정한 **FLUSHDB** 를 씁니다.

안전한 조회 명령

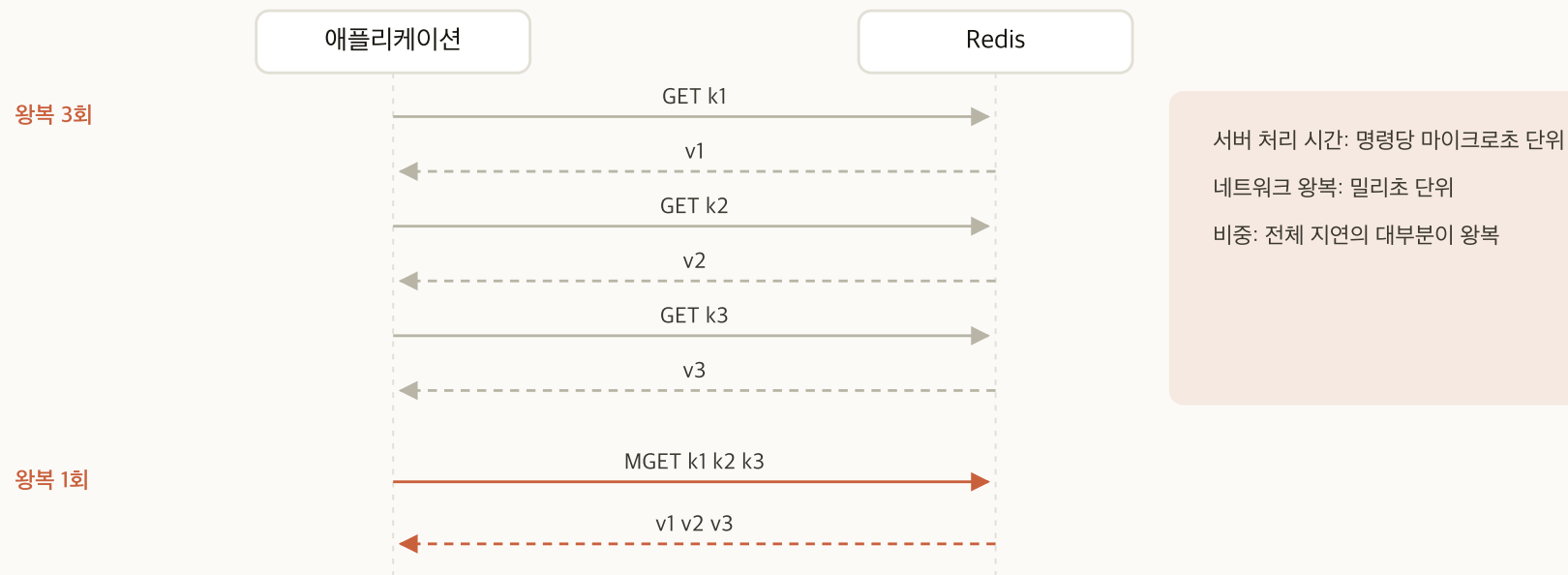
원래 명령	대체 명령	대체가 안전한 이유
KEYS	SCAN	커서를 이어 가며 조금씩 나눠 훑음. 한 번의 호출이 짧게 끝나 이벤트 루프를 오래 점유하지 않음
SMEMBERS	SSCAN	같은 방식으로 원소를 나눠 반환. 응답 하나가 작아짐
HGETALL	HSCAN 또는 HMGET	HSCAN 은 필드를 나눠 읽고, HMGET 은 필요한 필드만 지목해 애초에 적게 읽음

- **SLOWLOG** : 실제로 오래 걸린 명령을 사후에 확인하는 수단. `slowlog-log-slower-than` 의 단위는 마이크로초

나눠 읽는 명령이 총 작업량을 줄이지는 않습니다.

같은 양을 여러 번에 나눠 처리해 한 번의 점유 시간을 짧게 만드는 것입니다. 단일 스레드에서는 이 차이가 전부입니다.

지연의 주된 원인: 네트워크 왕복



서버 처리 시간: 명령당 마이크로초 단위
 네트워크 왕복: 밀리초 단위
 비중: 전체 지연의 대부분이 왕복

처리량이 부족하면 인스턴스를 확장하기 전에 왕복 횟수를 확인합니다.

Pipelining 은 응답을 기다리지 않고 명령 여러 개를 한 번에 보내는 방식이고, 왕복 횟수가 명령 수만큼 줄어듭니다. Pipelining과 다중 키 명령으로 왕복을 줄이는 것이 먼저입니다.

fill.sh 적재 스크립트

```
# ops 노드에서. 부하를 주는 쪽과 받는 쪽을 나눕니다
REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 64 1 # 64 MB 를 깊이 1 로 채움
```

인자	기본값	뜻
1번째	300	채울 크기(MB). 값 하나가 1 KiB 이므로 MB 당 키 1,024개
2번째	16	Pipelining 깊이. 응답을 기다리지 않고 몰아 보내는 명령 수. 1 이면 한 건씩 왕복

- **출력 세 덩이:** 적재 시작 줄, `redis-benchmark` 요약, 소요와 초당 건수
- **적재 시작 줄:** 목표 MB 에서 계산한 키 수와 요청 수. **요청이 키의 3배**라 같은 키를 세 번 덮어씀
- **초당 건수:** `appendfsync` 비교와 Eviction 재현에서 대조하는 값

appendfsync 비교에만 깊이 1 로 둡니다. 깊이 16 에서는 fsync 비용이 희석돼 정책 차이가 드러나지 않습니다.

기본 명령 확인 절차

- 1 ops 노드에서 `redis-cli -h 10.0.1.5` 실행
- 2 String·Hash·List·Set·Sorted Set·Stream을 하나씩 넣고 조회
- 3 셸에서 `REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 8` 로 8 MB, 약 8,000개를 채움
- 4 같은 데이터에 `KEYS *` 와 `SCAN 0 COUNT 10` 을 실행해 응답 방식 비교
- 5 `CONFIG SET slowlog-log-slower-than 0` 으로 모든 명령을 기록하게 함
- 6 명령을 몇 개 실행한 뒤 `SLOWLOG GET 5` 로 기록 확인
- 7 `CONFIG SET slowlog-log-slower-than 10000` 으로 되돌림

완료 판정

자료구조 6종 조회 성공, `SCAN` 이 `0` 이 아닌 커서를 돌려줌, `SLOWLOG GET` 에 항목이 나오면 완료입니다.

KEYS 와 SCAN 의 응답 차이

```
10.0.1.5:6379> KEYS *
1) "key:000000007778"
2) "key:000000005582"
... 7,818개를 한 번에
10.0.1.5:6379> SCAN 0 COUNT 10
1) "5632"
2) 1) "key:000000007778"
    2) "key:000000005582" ... 이번 호출에서 받은 것만
10.0.1.5:6379> SCAN 5632 COUNT 10
1) "5376"
```

커서가 0 으로 돌아올 때까지 나눠 받습니다.

KEYS 는 한 번의 응답에 전부 담고, SCAN 은 호출마다 일부와 다음 커서를 돌려줍니다.

04

Cache 패턴과 캐시 문제

Look-aside

Read-through

Write-through

Write-behind

Refresh-ahead

캐시 문제 4종

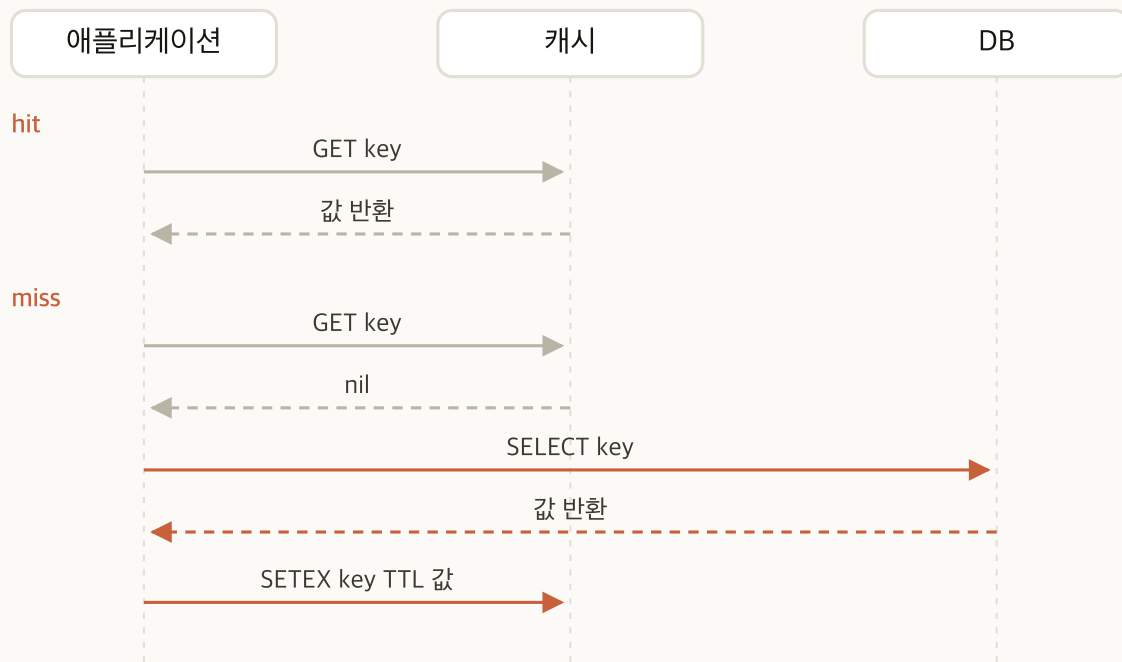
캐시의 역할

- **지연**: 원본 저장소보다 빠른 응답
- **부하**: 같은 조회를 원본에 반복하지 않음
- **비용**: 원본 저장소의 규모를 낮게 유지

캐시는 정합성을 성능과 바꾸는 장치입니다.

얼마나 오래된 데이터를 허용할 것인지가 캐시 설계의 첫 질문입니다.

Look-aside 패턴



- **주체:** 애플리케이션이 캐시와 DB를 모두 호출
- **hit 경로:** 캐시에서 바로 응답
- **miss 경로:** DB 조회 후 캐시에 적재

캐시가 정지해도 서비스는 유지됩니다.

대신 miss가 물리는 순간 DB에 부하가 그대로 넘어갑니다.

캐시 적재와 무효화

의사 코드

```
# 조회
v = redis.get(key)
if v is None:
    v = db.select(key)
    redis.setex(key, ttl, v)
return v

# 갱신
db.update(key, new_value)
redis.delete(key)          # 갱신이 아니라 삭제
```

갱신할 때는 캐시를 다시 쓰지 않고 지웁니다.

두 요청이 엇갈려 오래된 값이 캐시에 남는 경합을 줄일 수 있습니다.

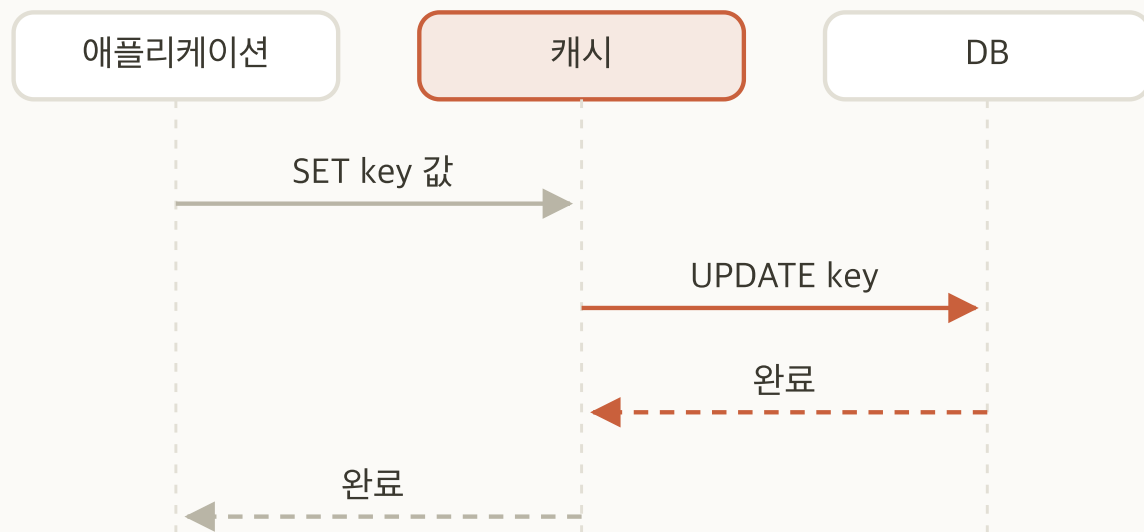
Read-through 패턴



- **주체**: 캐시 계층이 DB 조회를 대신함
- **hit 경로**: Look-aside와 같음
- **miss 경로**: 캐시 계층이 DB를 읽고 적재까지 마친 뒤 응답

코드는 단순해지지만 캐시 계층 장애가 곧 서비스 장애입니다.
 조회 경로가 캐시 계층을 지나가므로 우회로가 없습니다.

Write-through 패턴

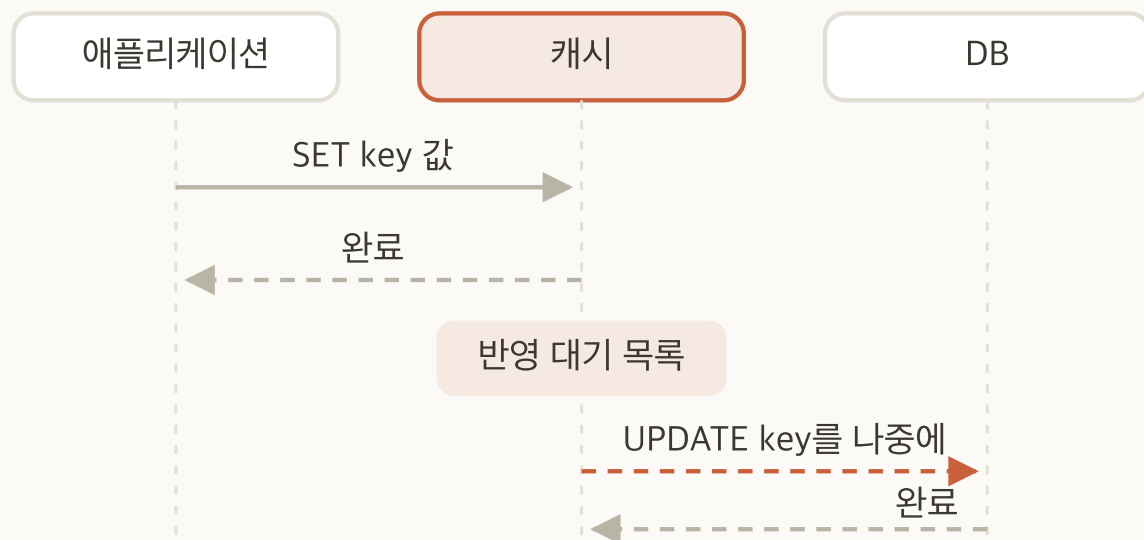


DB 반영이 끝나야 응답이 돌아감

- 동작: 쓰기를 캐시와 DB에 함께 반영
- 응답 시점: DB 반영이 끝난 뒤
- 결과: 캐시와 DB가 항상 같은 값

쓰기 지연이 DB의 응답 시간만큼 늘어납니다.
대신 캐시가 오래된 값을 들고 있는 구간이 없습니다.

Write-behind 패턴



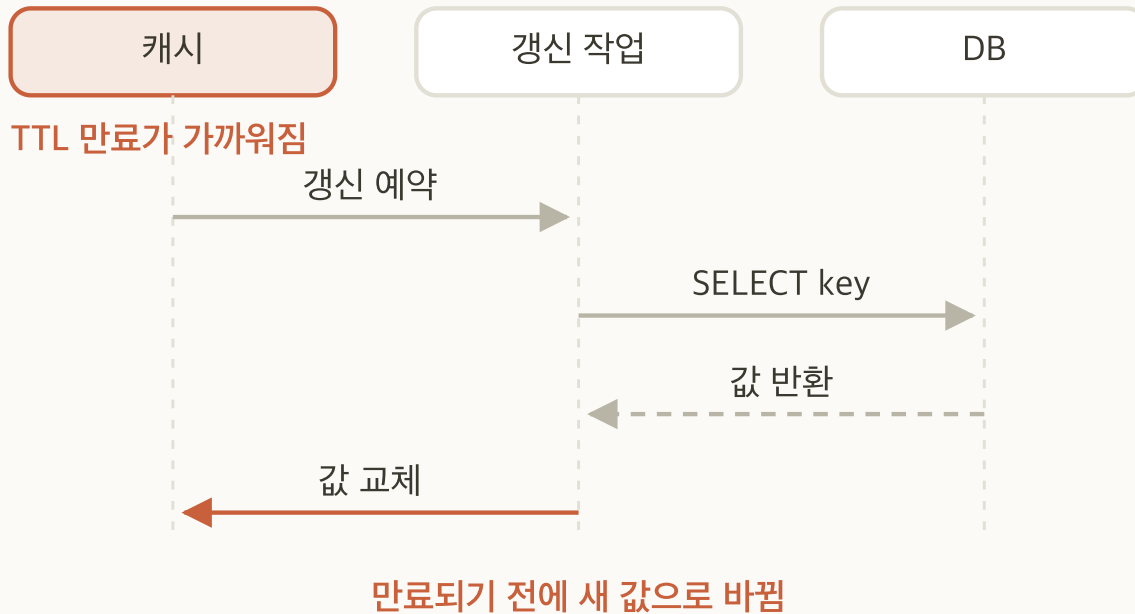
응답은 캐시에 쓴 직후 돌아감

- **동작:** 캐시에 먼저 쓰고 DB에는 나중에 반영
- **응답 시점:** 캐시에 쓴 직후
- **결과:** 쓰기가 빠른 대신 반영 대기 구간이 생김

반영되기 전에 캐시가 정지하면 그 쓰기는 사라집니다.

캐시 유실이 곧 데이터 유실이 되므로, 잃어도 되는 데이터에만 씁니다.

Refresh-ahead 패턴



- **전제:** 읽기 경로는 Look-aside를 그대로 따름. 갱신 시점만 앞당김
- **동작:** 만료가 가까워지면 미리 갱신
- **이득:** 사용자 요청이 miss를 겪지 않음
- **조건:** 어떤 키가 곧 쓰일지 예측할 수 있어야 함

접근 패턴이 예측 가능할 때만 유효합니다.

쓰이지 않을 키까지 미리 갱신하면 원본에 불필요한 부하만 늘어납니다.

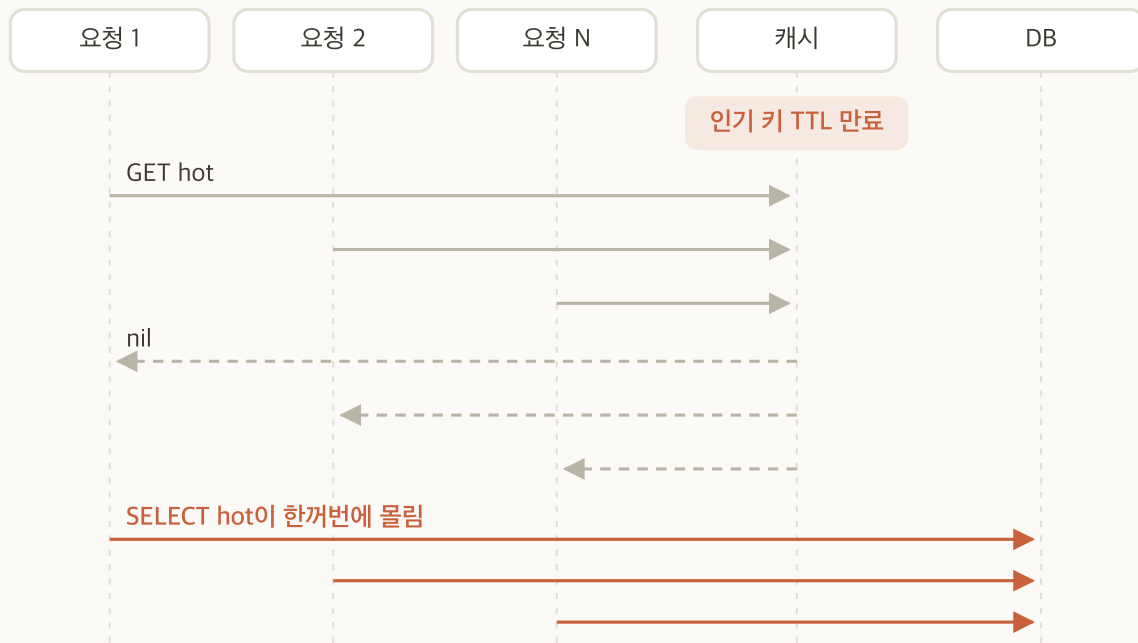
패턴 선택 기준

기준	Look-aside	Read-through	Write-through	Write-behind	Refresh-ahead
다루는 경로	읽기	읽기	쓰기	쓰기	읽기
데이터 최신성	TTL만큼 오래됨	TTL만큼 오래됨	항상 최신	항상 최신	만료 전 갱신
쓰기 지연	영향 없음	영향 없음	증가	감소	영향 없음
유실 위험	없음	없음	없음	있음	없음

이 세션의 설명은 Look-aside를 기준으로 합니다.
 가장 널리 쓰이는 Cache 패턴이고, 나머지 넷이 이 구조에서 무엇을 바꾼 것인지로 설명되기 때문입니다.

Refresh-ahead는 Look-aside 위에 얹는 갱신 방식이므로 읽기 경로가 Look-aside와 같습니다.

캐시 스탬피드 (Cache Stampede)

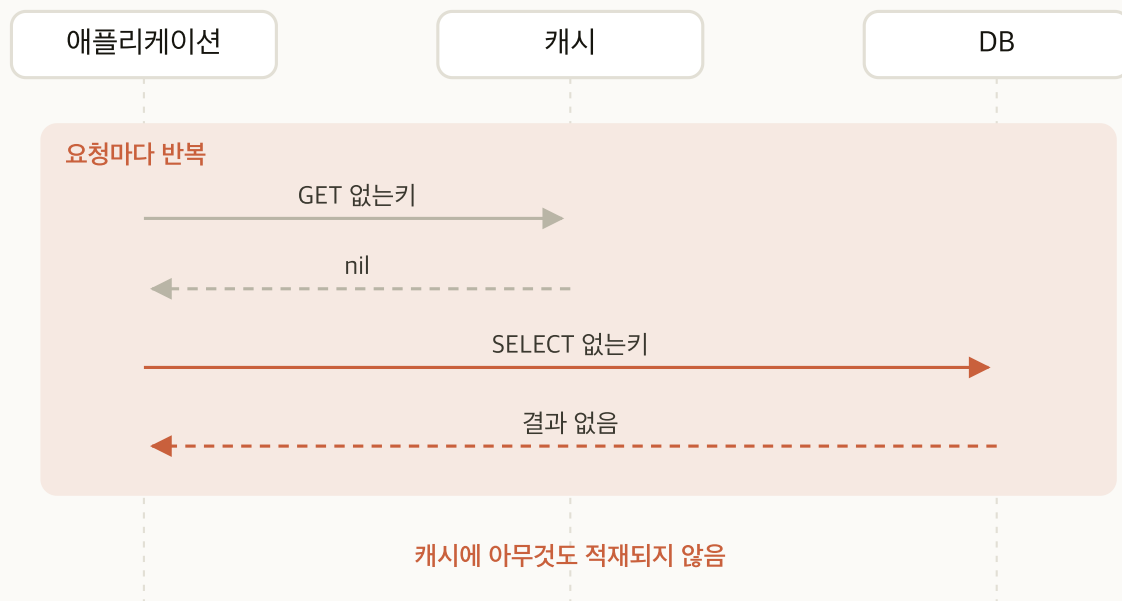


- **정의:** 인기 키 하나가 만료된 순간, 그 키를 찾던 요청이 한꺼번에 원본으로 몰리는 현상
- **원인:** 만료 시각이 한 점에 몰림
- **증상:** 특정 시각마다 DB 연결 수가 급증
- **대응:** TTL 분산(jitter), 단일 갱신자(Mutex), 논리적 만료

키 하나가 비었을 뿐인데 원본이 N배의 같은 질의를 받습니다.

첫 요청이 값을 채우기 전까지 뒤따르는 요청도 계속 miss 입니다. 한 요청만 원본을 부르게 하고 나머지는 그 결과를 기다리게 하는 것이 대응의 핵심입니다.

캐시 관통 (Cache Penetration)

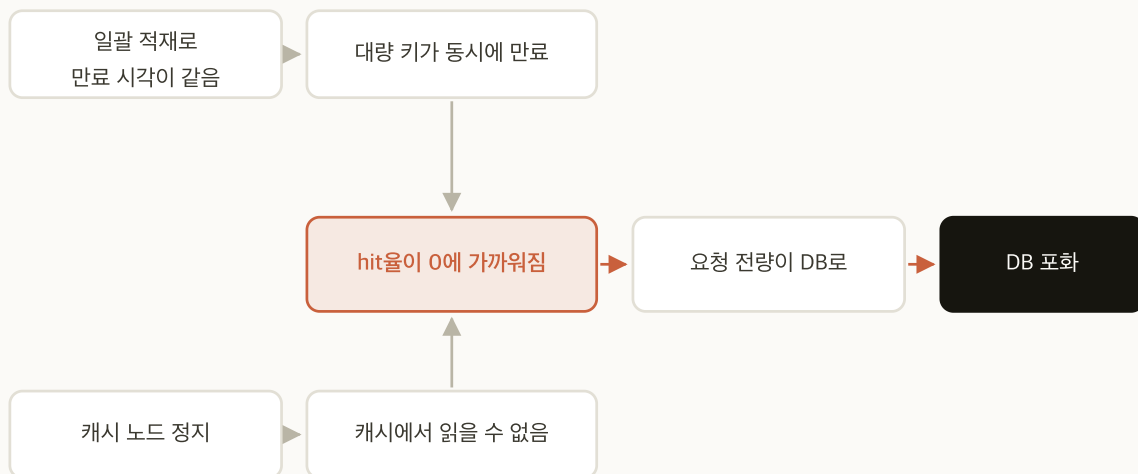


- **정의:** 캐시가 아무것도 막아 주지 못하고 요청이 매번 원본까지 도달하는 현상
- **원인:** 원본에도 없는 키를 반복 조회
- **증상:** 캐시 hit율이 낮는데 키 수는 늘지 않음
- **대응:** 음수 캐싱(짧은 TTL), Bloom filter로 사전 차단

없는 키도 캐시에 기록해야 막을 수 있습니다.

음수 캐싱의 TTL은 짧게 잡아 나중에 실제로 생긴 데이터를 오래 가리지 않게 합니다.

캐시 눈사태 (Cache Avalanche)



- **정의:** 대량의 키가 한꺼번에 사라져 캐시가 비어 있는 상태가 되고, 요청 전량이 원본으로 넘어가는 현상
- **원인:** 일괄 적재로 만료 시각이 같아짐, 또는 노드 상실
- **증상:** 캐시 hit율이 순간적으로 0에 가까워짐
- **대응:** 만료 시각 분산, 복제 구성, 원본 앞단의 유입 제한

캐시를 한꺼번에 적재하면 만료도 한꺼번에 일어납니다.
초기 적재 시점부터 TTL을 흘뜨려 두어야 합니다.

캐시 문제 비교

문제	원인 축	대응
스탬피드 (Cache Stampede)	만료 시각이 한 점에 몰림	TTL 분산, 단일 갱신자
관통 (Cache Penetration)	존재하지 않는 키	음수 캐싱, Bloom filter
눈사태 (Cache Avalanche)	규모, 대량 만료 또는 노드 상실	만료 분산, 다중화
일관성 붕괴 (Cache Inconsistency)	갱신과 무효화의 순서 경합	갱신 대신 삭제

앞의 세 가지는 증상이 비슷하지만 원인이 서로 다릅니다.
 대응도 각각 다르므로 「캐시 문제」로 뭉뚱그리지 않고 원인 축으로 구분합니다.

05

Persistence

RTO·RPO

RDB

fork와 Copy-on-Write

AOF

fsync 정책

적용 실습

Persistence의 목표: RTO와 RPO

구분	뜻	
RTO	복구 목표 시간. 재기동 후 서비스가 다시 응답하기까지	
RPO	허용 손실 구간. 장애 시점 기준 잃어도 되는 데이터의 시간 폭	
용도	RTO: 그동안 무엇이 안 되는가	RPO: 잃은 것을 되살릴 수 있는가
세션 저장소	재기동 동안 전원이 로그아웃 상태	없음. 잃은 구간의 사용자는 다시 로그인
조회 캐시	빈 채로 열면 원본에 부하가 몰림	원본에서 다시 채움. 제한 없음
순위·카운터	그 값을 쓰는 화면만 비어 있음	원본에서 다시 셀 수 있으면 몇 분까지
큐·작업 목록	쌓인 작업이 그동안 처리되지 않음	없음. 사실상 0

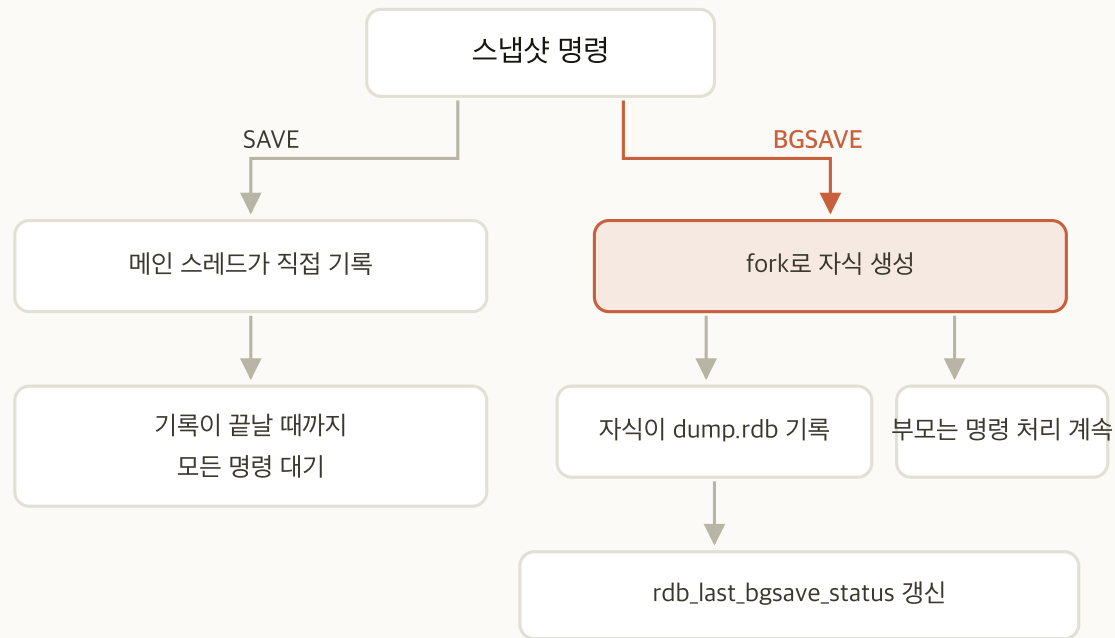
RDB: 스냅샷 방식

- **정의:** 어느 한 시점의 데이터 전체를 파일 하나로 저장하는 방식
- **생성 시점:** 설정한 조건을 만족할 때. 경과 시간과 변경된 키 수의 조합으로 정함
- **손실 구간:** 마지막 저장 시점 이후에 들어온 쓰기

복구가 빠른 대신 마지막 시점 이후를 잃습니다.

파일 하나만 읽어 들이면 되므로 재기동이 빠릅니다. 그 대가가 손실 구간입니다.

RDB 스냅샷

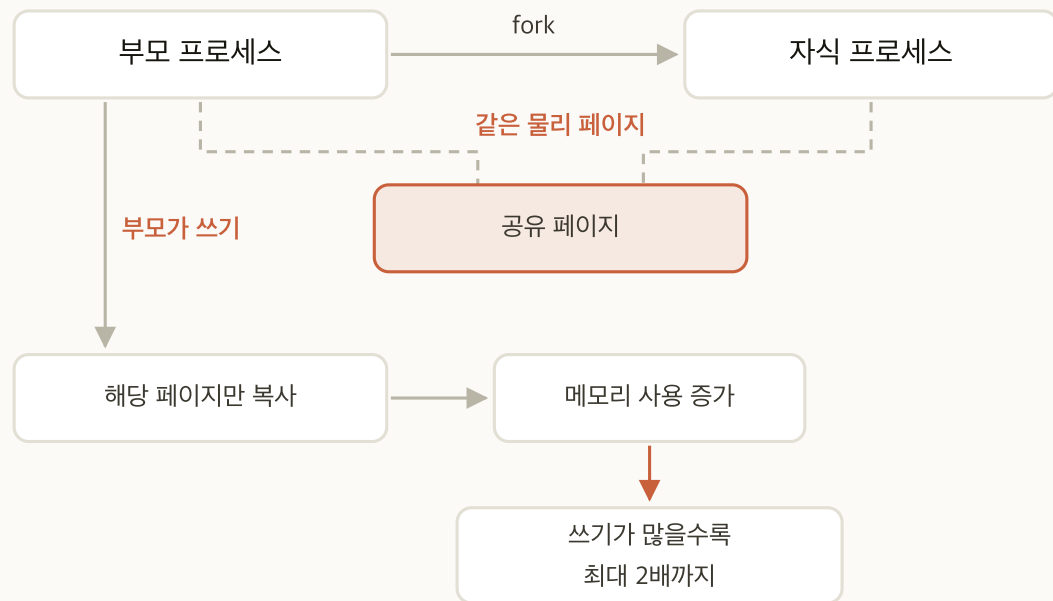


항목	내용
형식	특정 시점 전체 데이터의 바이너리 덤프
SAVE	메인 스레드에서 실행. 그동안 서버가 멈춤
BGSAVE	프로세스를 fork 해서 자식이 기록
장점	파일이 작고 복구가 빠름
단점	마지막 스냅샷 이후 데이터를 잃음

SAVE 는 운영 중에 쓰지 않습니다.

단일 스레드이므로 덤프가 끝날 때까지 모든 명령이 대기합니다.

fork와 Copy-on-Write



- **필요성**: 자식이 **fork 한 시점의 메모리를 그대로** 보아야 함. 부모의 쓰기가 자식에게 보이면 옛 값과 새 값이 섞임
- **시작**: 페이지를 함께 보므로 추가 메모리가 거의 없음
- **진행**: 부모가 값을 바꾸면 그 페이지만 복사. **부모는 새 사본, 자식은 원본**
- **최악**: 쓰기가 많으면 메모리 사용이 최대 2배까지 증가
- **vm.overcommit_memory**: 1로 두지 않으면 커널이 메모리 예약을 거부해 스냅샷이 실패할 수 있음

이것이 **maxmemory** 를 물리 메모리의 50%에서 60% 사이로 잡는 이유입니다.
남은 절반은 fork 한 자식이 쓰는 여유 공간입니다.

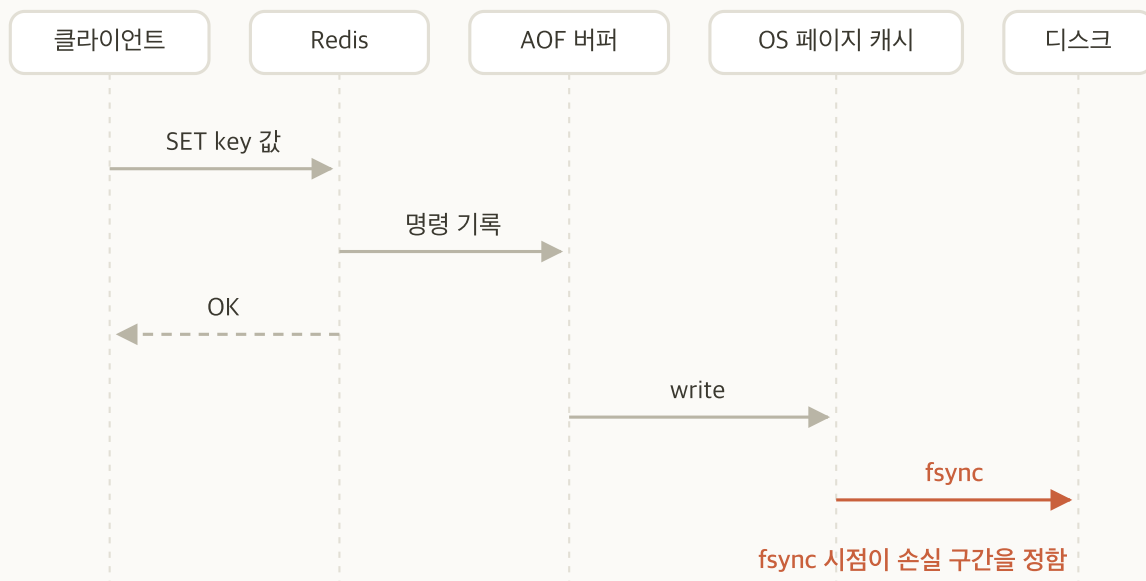
AOF: 로그 기록 방식

- **정의:** 데이터를 바꾸는 명령을 일어난 순서대로 파일에 덧붙이는 방식
- **복구 방법:** 기록된 명령을 처음부터 다시 실행
- **RDB와의 차이:** 결과가 아니라 과정을 남김

손실 구간이 짧은 대신 복구가 느립니다.

명령을 하나씩 재생해야 하므로 기록이 길수록 재기동이 오래 걸립니다.

AOF 기록 경로



항목	내용
형식	쓰기 명령을 순서대로 기록한 추가 전용 로그
복구	재기동 시 명령을 처음부터 재생
장점	손실 구간이 작음
단점	덧붙이기만 하므로 파일이 계속 커지고 복구가 느려짐

명령을 실행한 뒤에 기록합니다.
실패한 명령은 남지 않습니다.

fsync 필요성

- **write** 가 하는 일: 데이터를 OS 페이지 캐시까지만 넘김. 아직 디스크에 있지 않음
- 전원 차단 시: 페이지 캐시에만 있던 내용은 사라짐
- **fsync** 가 하는 일: 페이지 캐시의 내용을 디스크에 내려 쓰도록 지시

AOF에 기록했다는 것과 디스크에 남았다는 것은 다릅니다.
둘 사이의 간격을 얼마로 둘지가 곧 `appendfsync` 설정입니다.

fsync 정책 3종



정책	손실 구간	처리량
always	사실상 없음	가장 낮음
everysec	최대 1초	중간
no	OS가 내보낼 때까지. 예측 불가	가장 높음

손실 허용 구간과 처리량을 직접 맞추는 선택입니다.

기본값인 **everysec** 을 기준으로 두고, 1초 손실도 허용되지 않는 데이터인지 먼저 판단합니다.

AOF rewrite



- **필요성:** `incr` 는 덧붙이기만 하므로 계속 커짐. 재기동 때 재생할 명령이 늘어 복구가 점점 느려짐
- **역할:** 지금 메모리의 데이터셋을 그대로 만들어 내는 최소한의 명령 집합을 새로 씀
- **자동 실행 조건:** `auto-aof-rewrite-percentage` 100 과 `auto-aof-rewrite-min-size` 64mb 를 함께 만족할 때. 수동은 `BGREWRITEAOF`
- **대가:** fork 로 메모리가 늘고, 자식의 디스크 쓰기가 부모의 `fsync` 를 늦춤

rewrite 주기가 복구 시간의 상한을 정합니다.

재기동 때 재생하는 것은 마지막 rewrite 이후에 쌓인 `incr` 뿐입니다. 자주 돌수록 복구는 빨라지고 그만큼 fork 가 자주 일어납니다.

RDB와 AOF 조합

기록 부담과 복구 속도가 서로 반대로 움직입니다. RDB 는 전체를 한 번에 찍어 부담이 크고, 그래서 자주 찍지 못해 손실 구간이 커집니다. AOF 는 바뀐 명령만 이어 적어 부담과 손실 구간이 작은 대신 복구할 때 그것을 전부 재생해야 합니다.

구성	RTO	RPO	용도	고르는 조합
RDB만	빠름	마지막 스냅샷 시점까지	세션 저장소	혼합 (everysec)
AOF만 preamble 끄	느림. 명령 재생 필요	<code>appendfsync</code> 에 따름	조회 캐시	Persistence 끄 (셋 다 쓰지 않음)
혼합	빠름. RDB 부분 먼저 적재	<code>appendfsync</code> 에 따름	순위·카운터	RDB만
			큐·작업 목록	혼합 (always)

혼합은 둘을 서로의 약점에 붙인 것입니다.

복구는 RDB 형식의 base 를 적재해 끝내고, 손실 구간은 그 뒤의 AOF 가 줍니다.

Persistence 적용 절차

- 1 `redis-cli -h 10.0.1.5 INFO persistence` 로 현재 상태 확인
- 2 `ansible redis -a "redis-cli CONFIG SET appendonly yes"` 로 세 노드 모두 AOF 활성화
- 3 `REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 64 1` 로 적재하고 **출력된 처리량을 기록**
- 4 `aof_rewrite_in_progress` 가 0 이 될 때까지 기다린 뒤 `redis-cli -h 10.0.1.5 BGSAVE` . 1 이면 AOF rewrite 가 진행 중이어서 거부됩니다
- 5 `redis-cli -h 10.0.1.5 INFO persistence` 에서 결과와 AOF 파일 크기 확인
- 6 `ansible redis-1 -a "ls -l /var/lib/redis/appendonlydir" --become` 으로 파일 확인
- 7 `redis-cli -h 10.0.1.5 CONFIG SET appendfsync always` 로 바꾸고 **3단계와 같은 명령을 반복해 처리량 비교**
- 8 `REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 64` 로 한 번에 16개씩 보내며 한 번 더 측정
- 9 `redis-cli -h 10.0.1.5 CONFIG SET appendfsync everysec` 으로 되돌림

완료 판정 `rdb_last_bgsave_status:ok` 이고 `dump.rdb` 와 `appendonlydir/` 의 AOF 파일이 있으면 완료입니다.

`fill.sh` <채울 MB> <한 번에 보낼 명령 수> 는 1 KiB 짜리 키를 그만큼 채우고 처리량을 출력합니다. 둘째 인자의 기본값은 16 입니다. 3단계에서 1 을 주는 이유는 `always` 가 명령 하나마다가 아니라 한 번에 받은 묶음마다 `fsync` 하기 때문입니다. 16개씩 묶어 보내면 `fsync` 한 번을 16개가 나눠 부담해 `everysec` 과 차이가 나지 않습니다. 하나씩 보내야 정책 차이가 보입니다.

복구 확인

- 1 `redis-cli -h 10.0.1.5 DBSIZE` 로 지금 키 수를 기록. **DBSIZE** 는 그 DB 의 키 개수를 돌려줍니다
- 2 `redis-cli -h 10.0.1.5 BGSAVE` 로 스냅샷을 찍어 **RDB 시점을 고정**
- 3 `REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 16 1` 로 키를 더 넣고 **DBSIZE** 를 다시 기록
- 4 `ansible redis-1 -a "pkill -9 redis-server" --become` 로 **정상 종료 없이** 정지
- 5 `ansible redis-1 -m systemd -a "name=redis-server state=started" --become` 후 **DBSIZE** 확인. **3단계 값 그대로면 AOF 가 재생된** 것입니다
- 6 `CONFIG SET appendonly no` 후 4·5단계를 반복하고 **DBSIZE** 확인. **1단계 값으로 돌아가면 RDB 만 읽은 것입니다**
- 7 `CONFIG SET appendonly yes` 로 되돌림

5단계와 6단계의 차이가 RPO 입니다.

같은 방식으로 죽었는데 남은 데이터가 다릅니다. AOF 는 마지막 쓰기까지, RDB 는 마지막 스냅샷까지 살아남습니다.

정상 종료로는 이 차이가 보이지 않습니다. `systemctl stop` 은 종료 전에 AOF 와 RDB 를 마무리합니다. `FLUSHALL` 을 쓰지 않는 것도 같은 이유로, AOF 가 켜져 있으면 그 명령까지 기록되어 재기동해도 비어 있습니다.

명령과 예상 출력

「RDB와 AOF 조합」의 2·4·5단계를 노드 하나에 실행한 출력. 확인할 줄은 `rdb_last_bgsave_status` 와 `aof_enabled`

```
$ redis-cli -h 10.0.1.5 CONFIG SET appendonly yes
OK
$ redis-cli -h 10.0.1.5 BGSAVE
Background saving started
$ redis-cli -h 10.0.1.5 INFO persistence
# Persistence
rdb_last_bgsave_status:ok
aof_enabled:1
aof_last_write_status:ok
```


fsync 정책 비교

정책 (Pipelining 깊이 1)	처리량	Pipelining 깊이 (<code>always</code>)	처리량
<code>everysec</code>	95,919 건/초	1	17,546 건/초
<code>always</code>	17,546 건/초	16	142,910 건/초

정책	적용 대상
<code>always</code>	결제·주문·작업 큐처럼 한 건도 잃으면 안 되는 데이터
<code>everysec</code>	세션처럼 1초를 잃어도 되는 데이터. 대부분의 운영 구성
<code>no</code>	AOF는 켜 두되 손실 구간을 OS에 맡겨도 되는 데이터

`always` 의 비용은 한 번에 보내는 명령 수가 늘수록 줄어듭니다. 명령 하나마다 `fsync` 하는 것이 아니라 이벤트 루프 한 번마다 하기 때문입니다. 검증 환경에서 꽤 값이므로 디스크에 따라 달라집니다.

06

Eviction 정책

maxmemory

정책 8종

LRU

LFU

메모리 지표

적용 실습

maxmemory 필요성

- **전제:** Redis는 데이터를 메모리에 둬. 메모리는 유한함
- **한도 설정 시:** 한도에 닿았을 때 무엇을 버릴지 서버가 고름
- **한도 미설정 시:** 사용량이 계속 늘다가 OS의 OOM Killer가 프로세스를 종료
- **차이:** Eviction은 버릴 키를 고르고, OOM Killer는 종료할 프로세스를 고름

한도는 성능을 제한하는 값이 아니라 장애의 형태를 고르는 값입니다.

한도를 두면 「무엇을 버릴 것인가」가 되고, 두지 않으면 「프로세스 강제 종료」가 됩니다. 실습 환경에서도 반드시 설정합니다.

Redis 노드의 메모리 배분

항목	배분
물리 메모리	16 GiB
OS + exporter + Sentinel	1 GiB
Redis <code>maxmemory</code>	8 GiB
fork 여유	약 7 GiB

8 GiB는 물리 메모리의 절반입니다.

나머지 8 GiB 중 1 GiB는 OS와 exporter, Sentinel이 쓰고 약 7 GiB가 fork 여유로 남습니다. 쓰기가 몰리면 Copy-on-Write로 메모리 사용이 최대 2배까지 늘어납니다.

Eviction 필요성

- **상황:** `maxmemory` 에 닿았는데 새 쓰기가 들어옴
- **선택지:** 데이터 일부 삭제 또는 쓰기 거부
- **Eviction:** 무엇을 버릴지 서버가 고르는 규칙

버릴 것을 고르는 기준이 곧 정책입니다.

어떤 데이터가 없어져도 되는지는 용도가 정하므로, 정책 선택은 운영 판단입니다.

Eviction 정책 8종

정책	대상	고르는 기준
<code>noeviction</code>	없음	버리지 않고 쓰기를 거부
<code>allkeys-lru</code>	전체 키	가장 오래 사용되지 않은 키
<code>allkeys-lfu</code>	전체 키	사용 빈도가 가장 낮은 키
<code>allkeys-random</code>	전체 키	무작위
<code>volatile-lru</code>	TTL 있는 키	가장 오래 사용되지 않은 키
<code>volatile-lfu</code>	TTL 있는 키	사용 빈도가 가장 낮은 키
<code>volatile-random</code>	TTL 있는 키	무작위
<code>volatile-ttl</code>	TTL 있는 키	만료가 가장 가까운 키

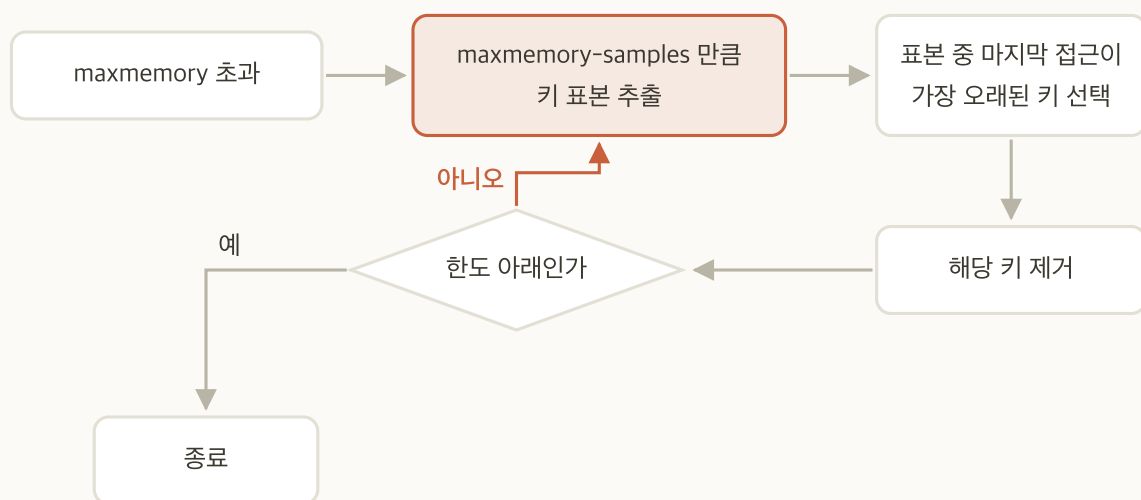
allkeys 와 volatile 의 선택

저장 데이터	고르는 정책	이유
조회 결과만 담은 캐시 전용 인스턴스	allkeys-lru	무엇이 버려져도 DB에서 다시 만들 수 있음
TTL 없는 설정값과 TTL 있는 캐시를 함께 담음	volatile-lru	캐시만 버려지고 설정값은 남음
작업 큐	noeviction	임의로 버리면 작업이 사라짐. 쓰기를 막는 편이 나음
인기 순위표	allkeys-lfu	최근성보다 꾸준한 접근 빈도가 중요함

volatile-* 로 설정했는데 TTL이 있는 키가 없으면 버릴 대상이 없습니다.

이때는 **noeviction** 과 같아져 쓰기가 거부됩니다.

LRU: 마지막 접근 시각

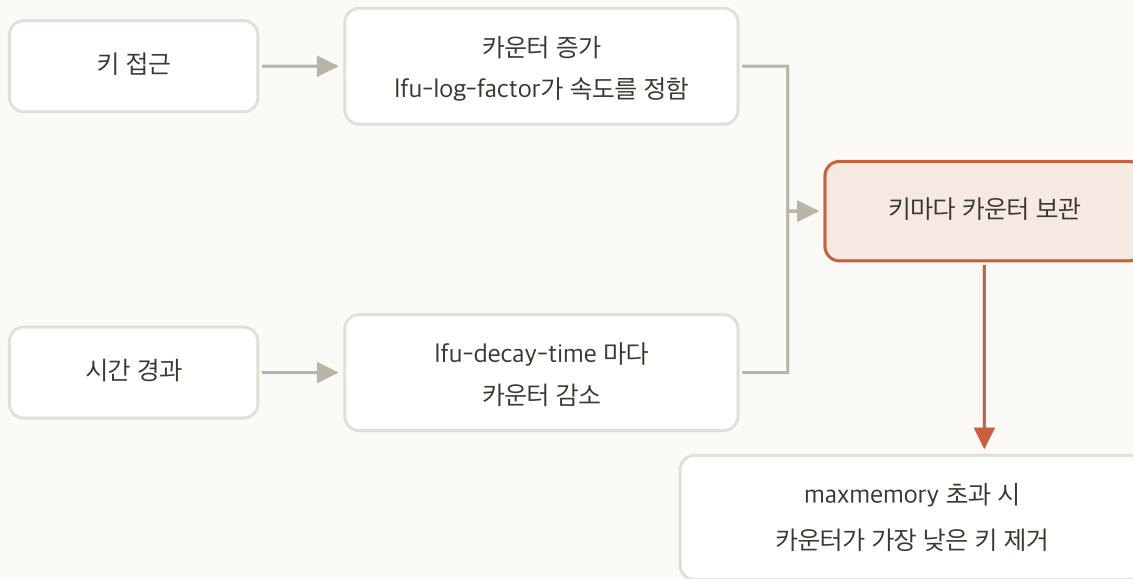


- **기준:** 마지막으로 접근한 시각. 오래된 것부터 버림
- **구현:** 전체를 정렬하지 않고 `memmemory-samples` 만큼 표본을 뽑아 그중 가장 오래된 키를 버림
- **기본값:** 표본 5개

정확한 LRU가 아니라 근사입니다.

전체 순서를 유지하는 데 드는 비용이 크므로 표본으로 대신합니다. 표본을 늘리면 선택이 정확해지지만 CPU 사용이 늘어납니다.

LFU: 접근 빈도와 감쇠



- **기준:** 접근 빈도. 사용 빈도가 가장 낮은 것부터 버림
- **감쇠:** 카운터가 시간이 지나면 줄어듦. 과거의 인기가 영원히 남지 않음
- **조절:** `lfu-log-factor` 로 증가 속도, `lfu-decay-time` 으로 감쇠 속도

일시적 트래픽 급증에도 캐시 내용이 덜 바뀝니다.

요청이 한 번 몰려도 카운터가 크게 오르지 않고, 오른 카운터도 시간이 지나면 줄어듭니다.

LRU와 LFU의 선택

기준	LRU	LFU
판단 근거	마지막 접근 시각	접근 빈도
일시적 트래픽 급증	그 키가 오래 남음	카운터가 조금 오르고 시간이 지나면 줄어듦
오래 사용되지 않은 인기 키	먼저 버려짐	카운터가 남아 늦게 버려짐
고르는 때	최근성이 중요한 데이터	꾸준한 인기가 중요한 데이터

접근 패턴이 시간에 따라 크게 바뀌면 LRU, 인기가 꾸준하면 LFU입니다.
 둘 다 `allkeys-` 와 `volatile-` 접두어를 붙여 대상 범위를 따로 정합니다.

noeviction 의 위험

- **동작:** 한도에 닿으면 쓰기 명령이 오류를 반환
- **오류:** `OOM command not allowed when used memory > 'maxmemory'`
- **읽기:** 계속 동작하므로 증상이 부분적으로만 보임

캐시 용도인데 **noeviction** 이면 그 자체가 장애입니다.

다만 큐나 세션 저장소처럼 임의로 버리면 안 되는 용도에서는 필요합니다. 용도가 정책을 정합니다.

Redis와 OS가 보는 메모리

지표	관측 대상
<code>used_memory</code>	Redis가 데이터에 쓰고 있다고 보는 양. 키와 값, 내부 자료구조의 합
RSS	OS가 프로세스에 실제로 내준 물리 메모리. 반납하지 않은 빈 공간도 포함

- `mem_fragmentation_ratio` : $RSS \div used_memory$. 1에 가까울수록 낭비가 없음
- 1보다 큼: 지운 자리가 OS로 돌아가지 않고 빈 조각으로 남은 상태. `activedefrag` 로 온라인 조각 모음 가능
- 1보다 작음: `used_memory` 만큼도 물리 메모리에 없는 상태. 일부가 디스크로 스왑됨

1보다 작은 쪽이 훨씬 위험합니다.

메모리 접근이 디스크 접근으로 바뀌어 응답 시간이 수백 배 늘어납니다.

이 세션에서 다루는 지표

메모리	의미	지속성과 복제	의미
<code>used_memory</code>	데이터가 쓰고 있는 메모리	<code>rdb_last_bgsave_status</code>	마지막 <code>BGSAVE</code> 결과
<code>mem_fragmentation_ratio</code>	RSS 대비 비율	<code>connected_slaves</code>	연결된 replica 수
<code>evicted_keys</code>	Eviction으로 버려진 키 수 누계	<code>sync_full</code>	replica 전체 재동기화가 일어난 횟수 누계
		<code>master_link_status</code>	replica가 보는 master 연결 상태

이 일곱 지표는 실습에서 값이 실제로 바뀝니다.

Persistence·Eviction·복제·자동 전환 네 구간에서 차례로 움직입니다.

maxmemory 와 Eviction 정책

- 1 `redis-cli -h 10.0.1.5 CONFIG SET maxmemory-policy noeviction` . 시작값은 `allkeys-lru`
- 2 `redis-cli -h 10.0.1.5 CONFIG SET maxmemory 256mb` 로 한도를 실습용으로 낮춤
- 3 `REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 300` 으로 한도를 넘길 만큼 적재
- 4 위 명령으로 8 MiB 값을 써서 오류 확인. 이어서 `GET user:1001:profile` 로 읽기 동작 확인
- 5 `redis-cli -h 10.0.1.5 CONFIG SET maxmemory-policy allkeys-lru` 로 되돌림
- 6 `REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 400` 으로 다시 적재. 앞서 채운 300보다 커야 새 키가 생김
- 7 `redis-cli -h 10.0.1.5 INFO stats` 의 `evicted_keys` 와 `INFO memory` 의 `used_memory` · `mem_fragmentation_ratio` 확인
- 8 `redis-cli -h 10.0.1.5 CONFIG SET maxmemory 8gb` 로 한도를 되돌림

256 MB는 실습 시간을 줄이려고 낮춘 값입니다.

이 노드의 실제 설정은 8 GiB이고 마지막 단계에서 되돌립니다. `noeviction` 에서 `OOM` 오류가 나고 `allkeys-lru` 로 바꾼 뒤 `evicted_keys` 가 증가하면 완료입니다.

noeviction 적용 결과

```
# 한도를 이미 넘긴 상태. 크기와 무관하게 거부된다
$ head -c 8388608 /dev/zero | tr '\0' 'x' | redis-cli -h 10.0.1.5 -x SET probe:big
OOM command not allowed when used memory > 'maxmemory'.
$ redis-cli -h 10.0.1.5 GET user:1001:profile
"{\"name\": \"김민준\", \"tier\": \"gold\"}"
```

- **head** 와 **tr** : 8 MiB(8,388,608 바이트) 만큼의 문자 **x** 를 만듦
- **redis-cli -x** : 마지막 인자를 표준 입력에서 읽음. **SET probe:big <8 MiB 문자열>** 이 됨

읽기는 계속 성공합니다.

쓰기만 막히므로 애플리케이션에서는 일부 기능만 실패하는 형태로 나타납니다.

allkeys-lru 적용 결과

evicted_keys 가 0에서 증가하고 used_memory 가 한도 아래로 유지됨

```
$ redis-cli -h 10.0.1.5 CONFIG SET maxmemory-policy allkeys-lru
OK
$ REDIS_HOST=10.0.1.5 ~/loadgen/fill.sh 400
소요 9.1초, 초당 134680건
$ redis-cli -h 10.0.1.5 INFO stats
# Stats
evicted_keys:630270
$ redis-cli -h 10.0.1.5 INFO memory
used_memory_human:255.68M
mem_fragmentation_ratio:1.06
```


07

복제

비동기 복제

명령 전파

full resync

백로그

읽기 분산

복제 구성

복제 필요성

- **단일 노드:** 그 노드가 멈추면 데이터와 서비스가 함께 사라짐
- **복제의 역할:** 같은 데이터를 다른 노드에 두어 하나가 멈춰도 남게 함
- **복제의 한계:** 담을 수 있는 데이터 양은 늘지 않음

복제는 가용성을 위한 장치입니다.

노드 하나를 잃어도 나머지가 같은 데이터를 들고 있으므로 서비스가 이어집니다.

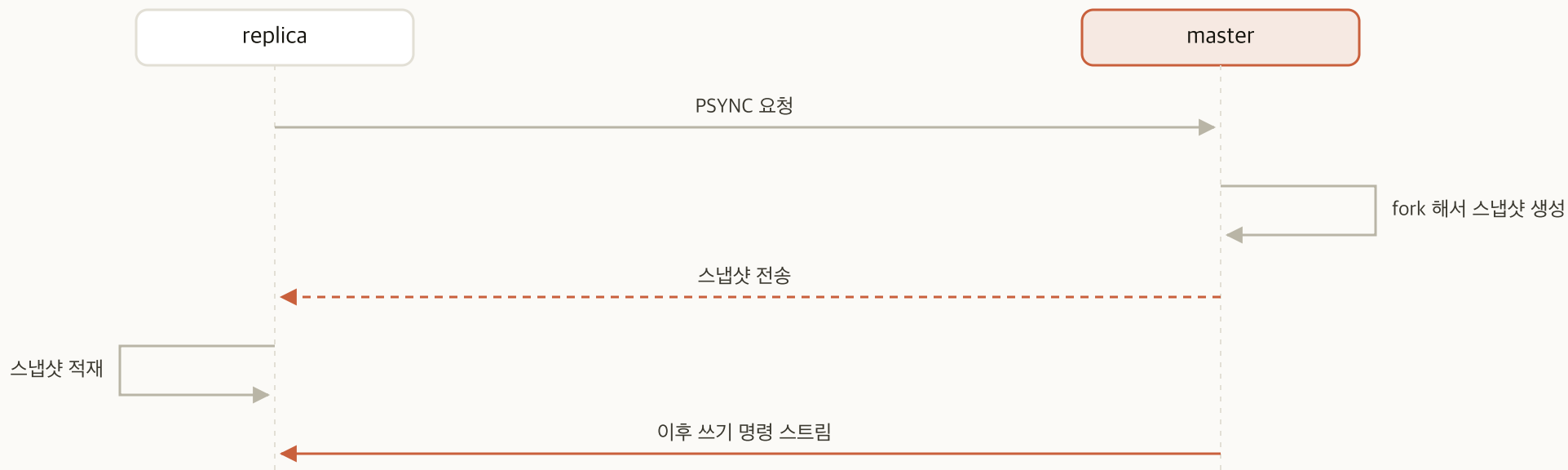
비동기 복제



- **응답 시점:** master는 replica의 반영을 기다리지 않음
- **이득:** 쓰기 지연이 복제 대상 수에 좌우되지 않음
- **대가:** master가 정지하는 순간 전파되지 않은 쓰기가 남음

전환 시 데이터가 일부 사라질 수 있습니다.
 설정 실수가 아니라 비동기 복제의 구조에서 나오는 결과입니다.

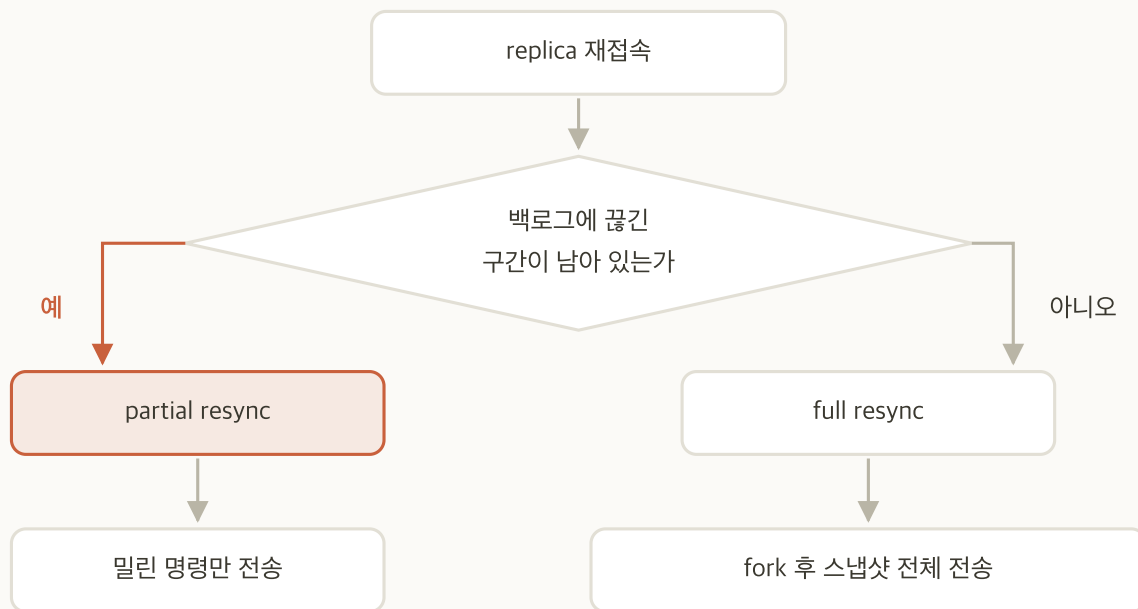
복제의 전파 방식



복제는 파일 복사가 아니라 명령 전파입니다.

최초 한 번만 스냅샷을 보내고, 이후에는 쓰기 명령을 계속 전송합니다.

full resync와 partial resync



구분	언제	master 부담
full resync	최초 접속, 백로그로 따라잡을 수 없을 때	fork와 전송 비용
partial resync	짧게 끊겼다가 재접속했고 백로그에 해당 구간이 남아 있을 때	미린 명령만 전송

replica 여러 대가 동시에 재접속하면 master에 부하가 집중됩니다.

백로그는 master 가 최근 쓰기 명령을 담아 두는 고정 크기 순환 버퍼입니다. full resync가 겹치면 fork와 전송이 한꺼번에 일어납니다. 실제 사고에서 자주 나오는 형태입니다.

복제 관련 설정

설정	역할	판단
<code>replicaof</code>	replica 로 지정할 master 주소. <code>REPLICAOF <IP> <port></code> 명령으로도 즉시 적용	설정 파일에 두면 재기동해도 유지됨
<code>repl-backlog-size</code>	partial resync용 순환 버퍼	크게 설정할수록 full resync 발생 확률이 낮아짐
<code>min-replicas-to-write</code>	연결된 replica 수가 부족하면 쓰기 거부	손실 위험을 가용성과 교환
<code>replica-read-only</code>	replica 의 쓰기 허용 여부. 기본값 yes	replica에 쓰면 master와 데이터가 어긋남
<code>replica-priority</code>	승격 우선순위	0이면 승격 대상에서 제외

읽기 분산의 대가

- **이득:** 읽기 부하를 replica로 나눔
- **대가:** replica가 보는 데이터는 master보다 오래됨
- **master 부담:** replica 대수만큼 같은 쓰기를 전송. 대수마다 출력 버퍼와 대역이 따로 듦
- **판단 기준:** 얼마나 오래된 데이터까지 허용할 것인가

읽기 분산은 정합성을 처리량과 바꾸는 선택입니다.

쓰기 직후 바로 읽는 경로는 master로 보내는 것이 안전합니다.

복제 구성

- 1 `redis-cli -h 10.0.1.6 REPLICAOF 10.0.1.5 6379`
- 2 `redis-cli -h 10.0.1.7 REPLICAOF 10.0.1.5 6379`
- 3 `redis-cli -h 10.0.1.5 INFO replication` 으로 `connected_slaves` 가 2 인 것 확인. **REPLICAOF 직후에는 0 이나 1 로 보일 수 있으니 다시 조회합니다**
- 4 `redis-cli -h 10.0.1.6 INFO replication` 에서 `master_sync_in_progress` 가 0 이고 `master_link_status` 가 `up` 인 것 확인
- 5 master에 키를 쓰고 replica에서 조회: `redis-cli -h 10.0.1.5 SET repl:t 1` 뒤 `redis-cli -h 10.0.1.6 GET repl:t` 가 1
- 6 replica에 쓰기를 시도해 거부 확인
- 7 `redis-cli -h 10.0.1.5 INFO stats` 에서 `sync_full` 이 2 인 것 확인

완료 판정

master가 `connected_slaves:2` , replica가 `master_link_status:up` 이고 `sync_full` 이 2 이상이면 완료입니다.

master의 INFO replication 출력

`state=online` 이어야 복제가 연결된 상태. `lag` 은 초 단위

```
$ redis-cli -h 10.0.1.5 INFO replication
# Replication
role:master
connected_slaves:2
slave0:ip=10.0.1.6,port=6379,state=online,offset=3259919,lag=0,io-thread=0
slave1:ip=10.0.1.7,port=6379,state=online,offset=3259919,lag=0,io-thread=0
```

replica의 INFO replication 출력

```
$ redis-cli -h 10.0.1.6 INFO replication
# Replication
role:slave
master_link_status:up
master_repl_offset:3259919
$ redis-cli -h 10.0.1.6 SET k v
(error) READONLY You can't write against a read only replica.
```

replica의 쓰기 거부는 정상 동작입니다.

이 보호가 없으면 replica에만 있는 데이터가 생겨 master와 내용이 달라집니다.

08

Sentinel과 Cluster

정의와 배치

네 가지 역할

quorum

failover 절차

3대 구성

Cluster 비교

Sentinel의 정의와 배치

- **정의:** Redis 배포판에 함께 들어 있는 별도 프로세스. 데이터를 담지 않음
- **하는 일:** master와 replica를 감시하고, master가 멈추면 replica를 승격
- **실행 방법:** `redis-sentinel` 로 기동. 기본 포트 26379
- **배치:** 이 구성에서는 데이터 노드 3대에 하나씩 함께 기동

Sentinel은 Redis 서버가 아닙니다.

키를 저장하지도, 클라이언트의 데이터 요청을 받지도 않습니다. 감시와 전환, 주소 안내만 합니다.

Sentinel 필요성

- **복제만 있을 때:** master가 멈춰도 replica가 자동으로 승격되지 않음
- **수동 전환:** 감지와 승격을 사람이 맡고 판단 기준이 담당자마다 다름
- **남는 문제:** 승격이 끝나도 클라이언트가 새 master 주소를 알아야 함

Sentinel은 승격을 대신하고 새 주소를 알려 줍니다.

둘 중 하나만으로는 서비스가 이어지지 않습니다. 승격이 끝나도 주소가 전달되지 않으면 애플리케이션은 정지한 노드로 계속 접속합니다.

Sentinel의 네 가지 역할

역할	내용
모니터링	master와 replica의 상태를 주기적으로 확인
알림	상태 변화를 외부에 통보
자동 failover	master 장애 시 replica를 승격
구성 제공자	클라이언트에 현재 master 주소를 제공

네 번째가 핵심입니다.

클라이언트는 master IP를 직접 지정하지 않고 Sentinel에 질의합니다. 그래서 전환 후에도 접속이 이어집니다.

SDOWN과 ODOWN

SDOWN

주관적 다운. Sentinel 하나가 `down-after-milliseconds` 동안 응답을 받지 못한 상태

ODOWN

객관적 다운. quorum 수만큼의 Sentinel이 같은 판단에 합의한 상태

- **SDOWN 사유:** 무응답만이 아님. `PING` 에 `+PONG` · `-LOADING` · `-MASTERDOWN` 이 아닌 응답이 와도 무응답으로 셈
- **ODOWN 은 master에만:** SDOWN 을 본 Sentinel 이 `SENTINEL is-master-down-by-addr` 로 나머지에 물음. replica 와 다른 Sentinel 은 합의 없이 SDOWN 까지만 판정

`down-after-milliseconds` 가 오탐과 전환 지연을 함께 정합니다.

짧게 잡으면 일시적 지연을 장애로 오인하고, 길게 잡으면 실제 장애를 늦게 감지합니다.

quorum과 majority

구분	결정 대상	정해지는 방식	이 구성의 값
quorum	master가 정지했다고 합의할 최소 Sentinel 수	운영자가 지정. 1 이상 Sentinel 총수 이하	2
majority	전환을 실제로 실행하기 위한 과반	Sentinel 총수 ÷ 2 + 1 로 자동 계산. 한 대를 잃고도 이 수가 남아야 하므로 3대가 최소	2

이 구성에서는 두 값이 모두 2입니다.

값이 같다고 개념이 같은 것은 아닙니다. quorum은 판정에, majority는 실행에 걸리는 조건입니다.

Sentinel 3대 구성의 근거

Sentinel 2대

master 노드가 정지하면 같은 노드의 Sentinel도 함께 정지. 남은 1대로는 과반 2를 채우지 못해 감지하고도 전환을 실행하지 못함

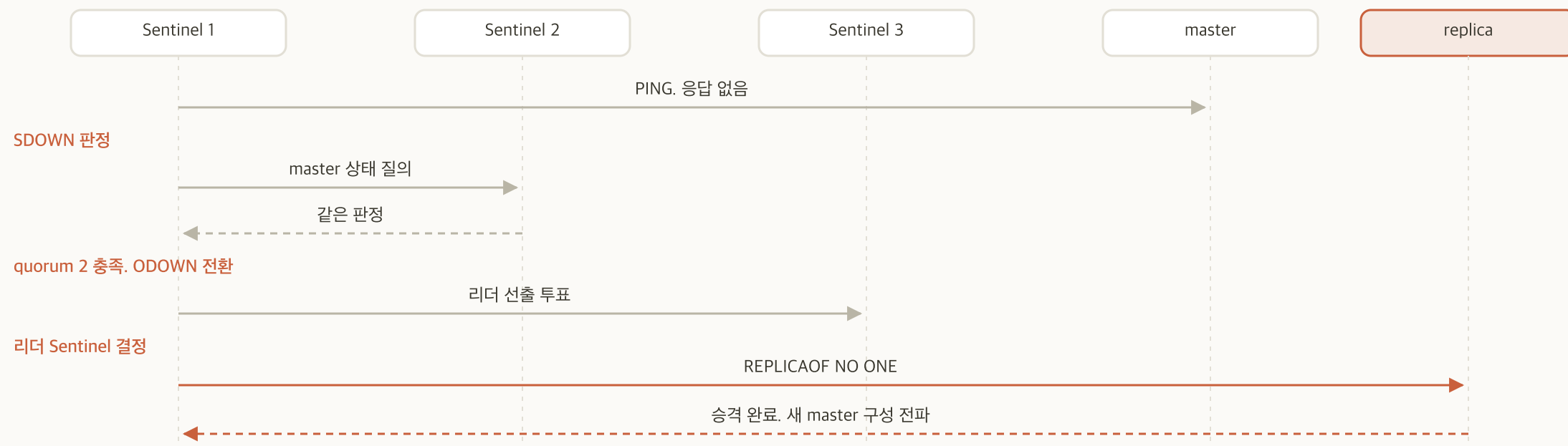
Sentinel 3대

한 노드가 정지해도 2대가 남음. quorum 2와 과반 2를 모두 채워 전환을 실행

Sentinel을 데이터 노드에 함께 배치합니다.

운영에서는 분리를 권하지만, 3대 구성에서 Sentinel 전용 노드를 3대 더 두는 것은 실습 규모를 넘습니다.

failover 절차



각 단계의 소요 시간이 모여 전체 전환 시간이 됩니다.

이후 실습에서 정지 시각부터 승격 완료까지를 직접 측정합니다.

승격 대상 선정 기준

순서	기준
1	<code>replica-priority</code> 가 낮은 노드 우선. 0이면 제외
2	복제 오프셋이 가장 앞선 노드
3	runid 사전순

특정 노드를 승격시키지 않으려면 `replica-priority` 를 0으로 둡니다.
 사양이 낮은 노드나 다른 리전의 노드를 제외할 때 씁니다.

원래 master의 복귀

- **재기동 시:** Sentinel이 replica로 재지정
- **데이터:** 새 master의 데이터로 맞춰짐
- **역할:** 자동으로 master로 돌아오지 않음

복귀 시점에 그 노드에만 있던 쓰기는 버려집니다.

비동기 복제의 결과입니다. 이 손실은 고장이 아니라 구조상 예정된 동작입니다.

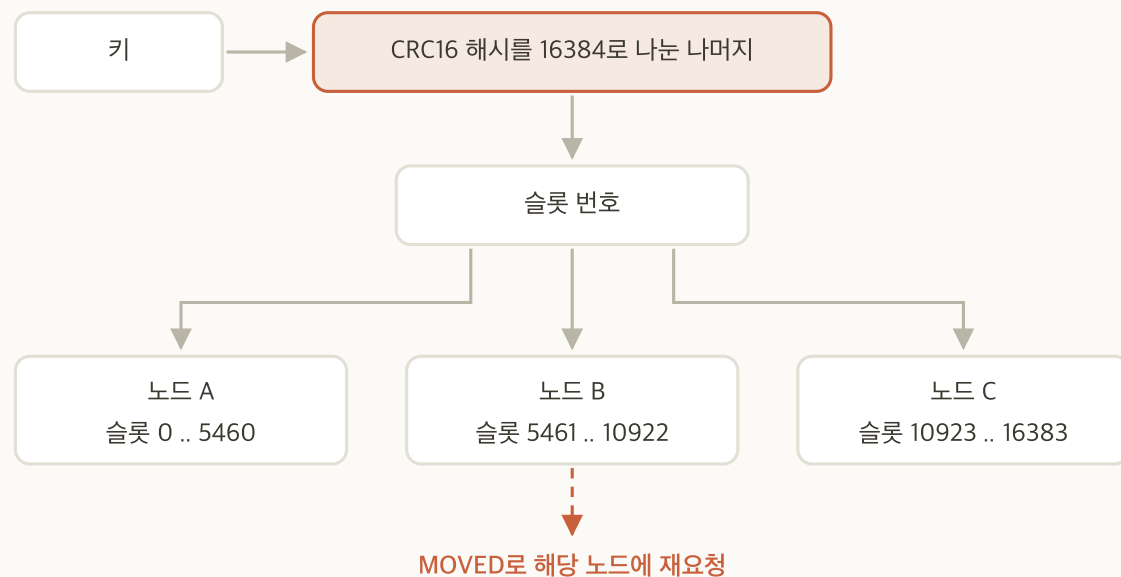
Cluster의 정의와 배치

- **정의:** 여러 Redis 노드가 데이터를 나눠 갖고 하나처럼 동작하는 구성
- **나누는 단위:** 해시 슬롯 16384개. 키마다 슬롯이 정해지고 슬롯마다 담당 노드가 있음
- **가용성:** 슬롯을 맡은 노드마다 replica를 둬. 노드가 멈추면 그 replica가 승격
- **별도 프로세스 없음:** Sentinel과 달리 감시자를 따로 두지 않고 노드끼리 서로 확인

Sentinel과 푸는 문제가 다릅니다.

Sentinel은 '멈춘 자리를 대신 세우는 것'이고, Cluster는 '한 대에 다 담기지 않는 데이터를 나누는 것'입니다.

Cluster의 슬롯 분할



- **MOVED**: 해당 슬롯이 다른 노드에 있음. 클라이언트가 그쪽으로 다시 요청
- **ASK**: 슬롯 이동 중. 이번 요청만 다른 노드로
- **멀티 키 연산**: 같은 슬롯에 있어야 함. 해시 태그 `{...}` 로 묶음

Sentinel과 Cluster의 차이

기준	Sentinel	Cluster
해결 대상	가용성	가용성과 수평 확장
데이터 배치	모든 노드가 전체 데이터 보관	슬롯 단위로 분할
용량 한계	단일 노드 메모리	노드 수에 비례
클라이언트	Sentinel에 master를 질의	슬롯 지도를 캐시하고 직접 접속
멀티 키 연산	제약 없음	같은 슬롯이어야 함

단일 노드 메모리 한계나 쓰기 처리량 한계에 닿을 때 Cluster를 검토합니다.
가용성만 필요하다면 Sentinel로 충분합니다.

sentinel.conf

주석을 뺀 설정 줄 전부

```
port {{ sentinel_port }}
bind {{ ansible_default_ipv4.address }} 127.0.0.1
protected-mode no
supervised systemd
dir /var/lib/redis-sentinel
logfile /var/log/redis/sentinel.log
sentinel monitor mymaster {{ redis_master_host }} {{ redis_port }} {{ sentinel_quorum }}
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 60000
sentinel parallel-syncs mymaster 1
```

- **왜 필요한가:** Sentinel 은 별개 데몬이라 무엇을 감시할지 스스로 알지 못함
- **배포 위치:** 세 Redis 노드의 `/etc/redis/sentinel.conf` . Ansible 플레이북이 배포함
- **mymaster** : 감시 대상에 **사람이 붙인 이름**. 세 노드가 같아야 함

sentinel.conf : 감시 대상

감시 대상 구간 발췌

```
##### 감시 대상 #####

# 마지막 숫자가 quorum
# quorum: master 가 정지했다고 합의할 최소 Sentinel 수
# 이 구성은 Sentinel 3대이므로 2 로 설정
sentinel monitor mymaster {{ redis_master_host }} {{ redis_port }} {{ sentinel_quorum }}
```

- **sentinel monitor** : 감시 대상과 quorum
- **마지막 인자**: quorum. 값은 인벤토리의 **sentinel_quorum** 에서 옴
- **mymaster** : 감시 대상에 붙인 이름. 클라이언트가 이 이름으로 질의

화면의 {{ }} 자리는 Ansible이 배포할 때 인벤토리 값으로 바뀝니다. Sentinel은 기동 뒤 이 파일을 스스로 갱신하므로 실제 노드의 파일은 화면의 템플릿과 내용이 다릅니다.

sentinel.conf : 타이밍

타이밍 구간 발췌

```
sentinel down-after-milliseconds mymaster 5000

# 전환을 다시 시도하기까지 기다리는 시간
sentinel failover-timeout mymaster 60000

# 새 master 에 동시에 재동기화할 replica 수
# 크게 잡으면 전환은 빨라지지만 그 순간 새 master 에 full resync 가 물림
sentinel parallel-syncs mymaster 1
```

- **down-after-milliseconds** : SDOWN 판정까지의 시간
- **failover-timeout** : 전환 재시도 간격
- **parallel-syncs** : 동시 재동기화 대수

이 실습에서는 값을 확인만 하고 바꾸지 않습니다.

Sentinel 기동과 확인

- 1 ops 노드에서 `ansible redis -m systemd -a "name=redis-sentinel state=started enabled=true" --become`
- 2 `ansible redis -a "systemctl is-active redis-sentinel"` 로 세 노드 기동 확인
- 3 `redis-cli -h 10.0.1.5 -p 26379 SENTINEL master mymaster` 로 감시 대상 확인. **Sentinel 끼리 서로를 인식하는 데 몇 초 소요.** `num-other-sentinels` 가 2 가 된 뒤 다음 단계로
- 4 `redis-cli -h 10.0.1.5 -p 26379 SENTINEL sentinels mymaster` 로 나머지 2대 인식 확인
- 5 `redis-cli -h 10.0.1.5 -p 26379 SENTINEL replicas mymaster` 로 replica 2대 확인
- 6 `redis-cli -h 10.0.1.5 -p 26379 SENTINEL get-master-addr-by-name mymaster` 로 클라이언트가 받는 주소 확인

복제가 없으면 Sentinel은 아무것도 하지 못합니다.

승격시킬 replica가 있어야 자동 전환이 성립하므로 복제를 먼저 구성하고 Sentinel을 기동합니다.

상태 확인 출력

```
$ redis-cli -h 10.0.1.5 -p 26379 SENTINEL master mymaster
1) "name"
2) "mymaster"
3) "ip"
4) "10.0.1.5"
...
9) "flags"
10) "master"
...
31) "num-slaves"
32) "2"
33) "num-other-sentinels"
34) "2"
35) "quorum"
36) "2"
```

09

master 정지와 자동 전환

정지

SDOWN

ODOWN

승격

복귀

리소스 정리

정지 실습의 목적

- **설정 검증:** 값이 적절한지는 실제로 정지시켜 보지 않으면 알 수 없음
- **전환 시간:** 직접 측정해야 장애 지속 시간을 수치로 제시할 수 있음
- **복귀 동작:** 정지했던 노드를 다시 올렸을 때 어떻게 편입되는지 확인

자동 전환은 한 번 실행해 보기 전까지 검증되지 않습니다.
설정값과 전환 시간, 복귀 동작은 정지를 재현해야 확인할 수 있습니다.

동작 확인 실습의 범위

- **조작**: master 프로세스 정지
- **관찰**: SDOWN → ODOWN → 승격 → 재지정 순서
- **측정**: 정지 시각부터 승격 완료까지의 소요 시간

무엇이 일어날지 미리 알고 시작합니다.

원인을 찾는 것이 아니라 구조가 앞에서 설명한 대로 동작하는지 확인합니다. 증상만 보고 원인을 찾는 실습은 2일차 장애대응 세션에서 합니다.

조작과 판정

- 1 `redis-cli -h 10.0.1.5 -p 26379 SENTINEL get-master-addr-by-name mymaster` 로 현재 master 주소 확인. **10.0.1.5** 와 **6379**
- 2 `redis-cli -h 10.0.1.5 SET loss:check 1` 로 손실 판정용 키를 미리 씀
- 3 `ansible redis-1 -m systemd -a "name=redis-server state=stopped" --become` 로 master 정지. **Sentinel** 은 별도 유닛이라 함께 멈추지 않음
- 4 `redis-cli -h 10.0.1.6 -p 26379 SENTINEL master mymaster` 로 SDOWN 확인. **flags** 에 **s_down** . 정지시킨 노드가 아닌 Sentinel 에 물어야 함
- 5 같은 명령을 다시 실행해 ODOWN 확인. **flags** 에 **s_down,o_down**
- 6 `redis-cli -h 10.0.1.6 -p 26379 SENTINEL get-master-addr-by-name mymaster` 로 승격 확인. 주소가 **10.0.1.6** 이나 **10.0.1.7** 로 바뀜
- 7 `redis-cli -h <새 master> INFO replication` 으로 재지정 확인. **connected_slaves:1**
- 8 `redis-cli -h <새 master> GET loss:check` 로 손실 판정. **값이 남아 있으면 손실 없음**

원래 master 복귀

- 1 `ansible redis-1 -m systemd -a "name=redis-server state=started" --become` 로 정지시킨 노드를 다시 기동
- 2 `redis-cli -h 10.0.1.5 INFO replication` 으로 역할 확인. `role:slave` . 처음에는 `role:master` 로 보일 수 있음
- 3 같은 출력의 `master_link_status` 확인. `up`
- 4 `redis-cli -h 10.0.1.6 -p 26379 SENTINEL replicas mymaster` 로 전체 확인. 원소 2개

원래 master는 자동으로 자리를 되찾지 않습니다.

replica로 합류합니다. 되찾게 하려면 `replica-priority` 로 승격 대상을 정한 뒤 `redis-cli -h 10.0.1.5 -p 26379 SENTINEL failover mymaster` 로 전환을 다시 일으켜야 합니다.

리소스 정리

```
terraform destroy -var-file=session.tfvars
```

- **확인:** 포털에서 리소스 그룹이 사라진 상태. `destroy` 가 한 번에 끝나지 않는 경우가 있어 반드시 포털에서 직접 확인

`destroy` 는 리소스 그룹과 그 안의 VM·디스크·네트워크를 전부 삭제합니다.
실행하면 되돌릴 수 없습니다.